# 第 1 章 逻辑代数的基础知识

逻辑代数是数字电路分析与设计的核心数学工具，贯穿于整个数字电子技术的学习过程。在深入学习逻辑代数的运算、公式和定理之前，首先补充数字电路的基本概念、数制与码制等核心知识点，为后续学习奠定基础。

## 一、数字电路与数字信号

### （一）数字电路的定义与特点

数字电路是专门处理数字信号的电子电路，其核心功能是对离散的数字信号进行运算、存储、传输和控制。与模拟电路相比，数字电路具有以下显著特点：一是信号离散性，数字信号仅取有限个确定值，最常用的是二值信号（0 和 1），分别对应电路中的低电平和高电平；二是抗干扰能力强，数字信号在传输过程中，只要干扰信号未超过阈值，就不会影响信号的正确识别，且便于通过整形电路恢复信号；三是精度高，数字电路的精度由位数决定，通过增加位数可灵活提高精度，而模拟电路受元件参数误差影响较大；四是易于集成，数字电路的基本单元（逻辑门）结构简单，可大规模集成在芯片中，实现复杂功能；五是便于存储和处理，数字信号可直接存储在寄存器、存储器中，且能通过逻辑运算实现算术运算、逻辑判断等功能。

数字电路广泛应用于计算机、通信、工业控制、家用电器、智能设备等领域，是现代电子技术的核心组成部分。而逻辑代数作为描述数字电路逻辑关系的数学工具，是分析和设计数字电路的理论基础，也称为布尔代数，由英国数学家乔治·布尔于 1847 年提出，后经美国科学家克劳德·香农于 20 世纪 30 年代应用于开关电路分析，逐步发展为适用于数字系统的逻辑代数体系。

### （二）模拟信号与数字信号的区别

自然界中的信号分为模拟信号和数字信号两类，二者的核心区别在于信号取值的连续性：模拟信号的取值是连续变化的，例如温度、压力、声音、电压等物理量的变化，其取值可以是任意实数；数字信号的取值是离散的，仅能取有限个确定值，最常用的是二值数字信号，仅包含 0 和 1 两种状态。

例如，传统温度计显示的温度值是模拟信号，其数值可以连续变化（如 25.1℃、25.15℃）；而数字温度计显示的温度值是数字信号，其数值是离散的（如 25℃、26℃），通过离散的数值近似表示连续的物理量。在电子电路中，模拟信号通常通过晶体管的放大作用处理，而数字信号则通过逻辑门的开关作用处理。

## 二、数制与码制

### （一）常用数制

数制是表示数值大小的计数方法，其核心是基数和位权。基数是数制中使用的数码个数，位权是数制中每一位数码所对应的权重，权重值为基数的幂次。数字电路中常用的数制有十进制、二进制、八进制和十六进制，其中二进制是数字电路的基础，八进制和十六进制用于简化二进制数的书写和表示。

1.十进制（Decimal）

基数为 10，使用的数码为 0~9，计数规则是“逢 10 进 1”。任意一个十进制数都可以表示为各位数码与其对应位权的乘积之和，其通用表达式为：

$$(N)_{10} = d_{n-1} \times 10^{n-1} + d_{n-2} \times 10^{n-2} + \cdots + d_1 \times 10^1 + d_0 \times 10^0 + d_{-1} \times 10^{-1} + \cdots + d_{-m} \times 10^{-m}$$

其中，$d_i$表示第 i 位的数码（0~9），$10^i$表示第 i 位的位权，n 为整数部分的位数，m 为小数部分的位数。例如，十进制数$(123.45)_{10}$的展开式为：

$$(123.45)_{10} = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}$$

2.二进制（Binary）

基数为 2，使用的数码为 0 和 1，计数规则是“逢 2 进 1”。二进制数的位权为 2 的幂次，其通用表达式为：

$$(N)_2 = b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \cdots + b_1 \times 2^1 + b_0 \times 2^0 + b_{-1} \times 2^{-1} + \cdots + b_{-m} \times 2^{-m}$$

其中，$b_i$表示第 i 位的数码（0 或 1），$2^i$表示第 i 位的位权。二进制数之所以被数字电路广泛采用，是因为其数码 0 和 1 恰好对应晶体管的截止与导通、开关的断开与闭合等两种对立的逻辑状态，易于用电子元件实现。例如，二进制数$(1011.01)_2$转换为十进制数的过程为：

$$\begin{aligned}(1011.01)_2 &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} \\ &= 8 + 0 + 2 + 1 + 0 + 0.25 = (11.25)_{10}\end{aligned}$$

3.八进制（Octal）

基数为 8，使用的数码为 0~7，计数规则是“逢 8 进 1”。八进制数的位权为 8 的幂次，其通用表达式为：

$$(N)_8 = o_{n-1} \times 8^{n-1} + o_{n-2} \times 8^{n-2} + \cdots + o_1 \times 8^1 + o_0 \times 8^0 + o_{-1} \times 8^{-1} + \cdots + o_{-m} \times 8^{-m}$$

由于 1 位八进制数恰好对应 3 位二进制数（$2^3 = 8$），因此八进制常用于简化二进制数的书写。例如，八进制数$(57.3)_8$转换为二进制数的过程为：将每一位八进制数转换为 3 位二进制数，不足 3 位补 0，即 5→101、7→111、3→011，最终得到$(101111.011)_2$。

4.十六进制（Hexadecimal）

基数为 16，使用的数码为 0~9、A~F（其中 A 对应 10、B 对应 11、C 对应 12、D 对应 13、E 对应 14、F 对应 15），计数规则是“逢 16 进 1”。十六进制数的位权为 16 的幂次，其通用表达式为：

$$(N)_{16} = h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \cdots + h_1 \times 16^1 + h_0 \times 16^0 + h_{-1} \times 16^{-1} + \cdots + h_{-m} \times 16^{-m}$$

由于 1 位十六进制数恰好对应 4 位二进制数（$2^4 = 16$），是数字系统中最常用的二进制简化表示方式，广泛应用于芯片引脚定义、程序编写、数据存储等场景。例如，十六进制数$(3A.5)_{16}$转换为二进制数的过程为：3→0011、A→1010、5→0101，最终得到$(00111010.0101)_2$，转换为十进制数为：

$$(3A.5)_{16} = 3 \times 16^1 + 10 \times 16^0 + 5 \times 16^{-1} = 48 + 10 + 0.3125 = (58.3125)_{10}$$

（二）不同数制之间的转换

1.二进制与十进制的转换

二进制转换为十进制：采用“按位权展开求和法”，即将二进制数的每一位数码乘以其对应的位权（2 的幂次），然后将所有乘积相加，得到对应的十进制数，具体示例见前文二进制部分。

十进制转换为二进制：分为整数部分和小数部分，分别采用不同的方法。整数部分采用“除 2 取余法”，即不断用十进制整数除以 2，记录每次的余数，直到商为 0，然后将余数从后往前排列，得到二进制整数部分；小数部分采用“乘 2 取整法”，即不断用十进制小数乘以 2，记录每次整数部分的数值，直到小数部分为 0（或达到所需精度），然后将整数部分从前往后排列，得到二进制小数部分。

示例：将十进制数$(25.625)_{10}$转换为二进制数。

整数部分（25）：25÷2=12 余 1；12÷2=6 余 0；6÷2=3 余 0；3÷2=1 余 1；1÷2=0 余 1，余数从后往前排列，得到整数部分为 11001。

小数部分（0.625）：0.625×2=1.25 取整 1；0.25×2=0.5 取整 0；0.5×2=1.0 取整 1，小数部分为 0，整数部分从前往后排列，得到小数部分为 011。

因此，$(25.625)_{10}=(11001.011)_2$。

2.二进制与八进制、十六进制的转换

二进制转换为八进制：将二进制数从右往左，每 3 位分为一组（小数部分从左往右，每 3 位分为一组），不足 3 位的补 0，然后将每组 3 位二进制数转换为对应的 1 位八进制数，即可得到八进制数。

二进制转换为十六进制：将二进制数从右往左，每 4 位分为一组（小数部分从左往右，每 4 位分为一组），不足 4 位的补 0，然后将每组 4 位二进制数转换为对应的 1 位十六进制数，即可得到十六进制数。

八进制、十六进制转换为二进制：与上述过程相反，将每 1 位八进制数转换为 3 位二进制数，每 1 位十六进制数转换为 4 位二进制数，不足位数补 0，然后将所有二进制数依次连接，即可得到二进制数。

（三）码制

码制是指用二进制数表示十进制数、字母、符号等信息的编码方式。在数字系统中，不仅需要处理数值信息，还需要处理文字、符号等非数值信息，因此需要制定统一的编码规则，常用的码制有 BCD 码、ASCII 码等。

(1) BCD 码（二-十进制编码）

BCD 码是用 4 位二进制数表示 1 位十进制数（0~9）的编码方式，其核心是“每 1 位十进制数对应 4 位二进制数”，但 4 位二进制数的 16 种组合（0000~1111）中，仅使用前 10 种组合（0000~1001）表示 0~9，后 6 种组合（1010~1111）为无效码。常用的 BCD 码有 8421 码、5421 码、余 3 码等，其中 8421 码是最常用的 BCD 码，其 4 位二进制数的位权分别为 8、4、2、1，与二进制数的位权一致，例如十进制数 5 对应的 8421 BCD 码为 0101，十进制数 9 对应的 8421 BCD 码为 1001。

需要注意的是，BCD 码与二进制数的表示方式不同，例如十进制数 10，对应的 8421 BCD 码为 00010000（两位 BCD 码），而对应的二进制数为 1010，二者不可混淆。

(2) ASCII 码（美国信息交换标准代码）

ASCII 码是用于表示字符（字母、数字、符号）的编码方式，采用 7 位二进制数表示，共可表示 128 种字符，其中包括 0~9 数字、26 个大写英文字母、26 个小写英文字母、标点符号、控制字符等。例如，数字 0 对应的 ASCII 码为 00110000（十进制 48），大写字母 A 对应的 ASCII 码为 01000001（十进制 65），小写字母 a 对应的 ASCII 码为 01100001（十进制 97）。在实际应用中，通常将 7 位 ASCII 码扩展为 8 位（最高位补 0），便于在 8 位数字电路中传输和处理。

**三、数字电路的发展、分类与学习方法**

1.数字电路的发展

数字电路的发展与半导体、集成电路技术深度绑定：20 世纪 60 年代进入小规模集成电路（SSI）时代；70 年代发展到中规模集成电路（MSI），出现标准化的译码器、计数器等功能器件；80 年代进入大规模（LSI）、超大规模（VLSI）集成电路时代，微处理器、存储器、可编程逻辑器件快速普及；21 世纪后进入片上系统（SoC）时代，单芯片可集成数十亿个晶体管，实现完整数字系统功能，FPGA/CPLD 等可编程器件让数字系统设计更灵活高效。

2.数字电路的分类

按电路结构和工作原理，数字电路分为两大类：

组合逻辑电路：任一时刻的输出仅取决于该时刻的输入组合，与电路历史状态无关，无存储单元、无记忆功能，典型电路有编码器、译码器、加法器、数据选择器等。

时序逻辑电路：任一时刻的输出不仅取决于该时刻的输入，还与电路历史状态有关，包含触发器、寄存器等存储单元，具备记忆功能，典型电路有锁存器、触发器、移位寄存器、计数器等。

按核心器件类型，可分为双极型电路（TTL、ECL 等，速度快、驱动能力强）和单极型 MOS 电路（CMOS 为主，功耗极低、集成度高，是当前大规模数字集成电路的主流工艺）。

3.课程内容与学习方法

本课程是电子信息、自动化、计算机等专业的核心技术基础课，核心内容包括逻辑代数基础、组合逻辑电路、时序逻辑电路，延伸讲解脉冲波形产生与整形、数模/模数转换、半导体存储器、可编程逻辑器件等。

学习核心要点：

- 抓住逻辑关系这一核心，数字电路的本质是实现逻辑功能，而非关注电压电流的数值细节；
- 熟练掌握逻辑代数这一数学工具，是分析和设计数字电路的基础；
- 理论与实践结合，通过仿真、实验、硬件设计加深对电路的理解；
- 建立模块化设计思维，复杂数字系统由基础功能模块组合而成，需掌握各类模块的功能与使用方法。

## 1.1 逻辑代数中的逻辑运算、公式和定理

逻辑代数（Logic Algebra），又称布尔代数（Boolean Algebra），是由英国数学家乔治·布尔（George Boole）于 1847 年在其著作《逻辑的数学分析》中首次提出的数学体系，最初用于解决形式逻辑中的演算问题。直到 1938 年，美国科学家克劳德·香农（Claude Shannon）将布尔代数系统应用于继电器开关电路的分析与设计中，这一数学工具才正式成为数字电路与数字系统分析、设计的核心数学基础，贯穿了数字逻辑电路从底层门电路设计到复杂数字系统搭建的全流程。

与普通初等代数不同，逻辑代数的研究对象是逻辑关系而非数值运算，其核心是通过约定的符号与规则，描述客观世界中“条件”与“结果”之间的因果逻辑关系。在逻辑代数体系中，所有的逻辑变量仅存在逻辑 0 和逻辑 1 两种互斥的取值，这里的 0 和 1 并不代表任何数值大小、数量多少，也不具备正负属性，仅用来表征一对完全对立的逻辑状态——比如事件的“不发生”与“发生”、开关的“断开”与“闭合”、电平的“低电平”与“高电平”、命题的“假”与“真”。

在数字系统的工程应用中，行业内普遍约定两种逻辑电平赋值体制：一种是正逻辑体制，用高电平表示逻辑 1，低电平表示逻辑 0；另一种是负逻辑体制，用低电平表示逻辑 1，高电平表示逻辑 0。两种逻辑体制对同一电路的逻辑功能描述存在对应转换关系，正逻辑体制下的

与运算对应负逻辑体制下的或运算，正逻辑体制下的或运算对应负逻辑体制下的与运算，非运算在两种体制下的逻辑功能保持不变。本章节若无特殊说明，所有逻辑描述均采用正逻辑体制。

### 1.1.1　三种基本逻辑运算

逻辑代数中的每一种逻辑运算，都对应着一种能够实现该逻辑功能的基本单元电路，这类电路统称为逻辑门电路，简称门电路。门电路是构成所有数字集成电路的最小硬件单元，如同搭建数字大厦的基础砖块，而逻辑运算的规则，就是门电路输入与输出之间必须遵循的硬件逻辑关系。为了便于工程设计、学术交流与教学应用，每一种门电路都有统一规定的标准逻辑符号，本节将结合每种逻辑运算的核心规则，同步介绍其对应的逻辑符号、硬件实现电路与工程应用，所有常用逻辑门的标准符号汇总见图 1-1-1。

逻辑代数中最基础、不可再拆分的逻辑运算为与运算、或运算、非运算三种，也被称作三种基本逻辑运算。自然界与工程领域中所有复杂的因果逻辑关系，都可以拆解为这三种基本逻辑的组合形式，因此这三种运算也是搭建所有数字逻辑电路的底层基石，所有数字集成电路的功能实现，最终都可以追溯到这三种基本逻辑运算的组合与叠加。

图 1-1-1　常用逻辑门标准符号汇总（国标 GB/T 4728）

#### 一、与运算（逻辑乘运算）

1.　核心定义

与逻辑是工程中最常见的因果逻辑之一，我们可以通过一个直观的生活电路案例，清晰理解与逻辑的本质，如图 1-1-2 所示：在一个由直流电源、两个相互串联的开关 A、开关 B，以及一个负载灯泡$Y_1$共同组成的闭合回路中，灯泡能否点亮，完全取决于两个开关的状态。只有

当开关 A 完全闭合、同时开关 B 也完全闭合时，整个回路才能导通，灯泡$Y_1$才会被点亮；只要两个开关中有任意一个处于断开状态，整个回路就会处于断路状态，灯泡$Y_1$必然无法点亮。

图 1-1-2　与逻辑电路（串联开关电路）

在这个典型的因果关系中，“开关闭合”是决定事件结果的前置条件，“灯泡点亮”是事件的最终结果，我们可以从中提炼出与逻辑的通用核心定义：只有当决定某一事件结果的全部条件，都同时得到满足时，该事件的结果才会最终发生；只要有任意一个条件不满足，事件结果就不会发生。这种“全条件满足才触发结果”的因果逻辑关系，就被称为与逻辑，与之对应的数学运算，就是与运算，也被称作逻辑乘运算。

2.　逻辑表达式

与运算的通用逻辑表达式为：

$$Y = A \cdot B$$

在该表达式中，A、B 为输入逻辑变量，代表决定事件结果的两个前置条件；Y 为输出逻辑变量，代表事件的最终结果；符号“·”为与运算的运算符号，读作“与”，也可读作“逻辑乘”。在不引起逻辑歧义的前提下，与运算的符号“·”可以直接省略，表达式可简写为$Y = AB$，其书写形式与普通初等代数中的乘法书写一致，但二者的运算规则与物理含义存在本质区别，不可混为一谈。

当逻辑事件存在多个输入条件时，与运算可以直接扩展至多变量形式，n 个输入变量的与运算表达式为：

$$Y = A \cdot B \cdot C \cdot \ldots \cdot N$$

同样可简写为：

$$Y = ABC \ldots N$$

3.　真值表

真值表是逻辑代数中用于表征输入逻辑变量与输出逻辑结果之间，所有取值对应关系的标准化表格，也是逻辑电路分析与设计中最基础、最直观的工具。对于一个拥有 n 个输入变量的逻辑运算，共有$2^n$种不同的输入取值组合，真值表会将所有输入组合对应的输出结果全部无遗漏地罗列出来，确保逻辑关系的完整性与唯一性。

对于双输入变量的与运算，共有$2^2 = 4$ 种输入取值组合，其完整真值表如下表所示：

表 1-1-1　与逻辑真值表

| 输入逻辑变量 A | 输入逻辑变量 B | 输出逻辑变量 Y=AB | 对应逻辑含义 |
|---|---|---|---|
| 0 | 0 | 0 | 两个条件均不满足，事件不发生 |
| 0 | 1 | 0 | 条件 A 不满足、条件 B 满足，事件不发生 |
| 1 | 0 | 0 | 条件 A 满足、条件 B 不满足，事件不发生 |
| 1 | 1 | 1 | 两个条件均满足，事件发生 |

4.　核心运算规则

从上述真值表中，我们可以提炼出与运算的通用核心运算规则，也被称作与运算口诀：有 0 出 0，全 1 出 1。

该规则可拆解为两层核心含义：一是当所有输入逻辑变量中，存在任意一个变量取值为逻辑 0 时，无论其他输入变量的取值是逻辑 0 还是逻辑 1，最终的与运算输出结果必然为逻辑 0；二是只有当所有输入逻辑变量的取值，全部为逻辑 1 时，最终的与运算输出结果才会为逻辑 1。

基于该核心规则，我们可以直接推导出与运算的基础运算公式：

$$0 \cdot 0 = 0、0 \cdot 1 = 0、1 \cdot 0 = 0、1 \cdot 1 = 1$$

该规则同样适用于多变量与运算，例如四变量与运算中，$1 \cdot 0 \cdot 1 \cdot 1 = 0$，仅当 $1 \cdot 1 \cdot 1 \cdot 1 = 1$，其余所有包含逻辑 0 的输入组合，输出结果均为逻辑 0。

5.　与门的电路实现与工程应用

实现与运算的门电路称为与门，二输入与门的标准逻辑符号见图 1-1-1，符号的输入端对应逻辑变量 A、B，输出端对应运算结果 Y。

二输入与门可通过基础的分立元件搭建，核心由两个硅二极管和一个限流电阻构成。电路中两个二极管的阳极共接，经限流电阻接至+5 V 正电源，两个二极管的阴极分别作为电路的两个信号输入端，阳极公共端作为电路的信号输出端，如图 1-1-3 所示。

图 1-1-3　二极管与门电路

其工作过程完全契合与运算的核心规则：当任意一个输入端接入低电平时，对应的二极管获得正向偏置而导通，输出端电位被钳位在低电平，对应逻辑 0；只有当两个输入端全部接入

高电平时，两个二极管均处于反向截止状态，限流电阻无电流通过，输出端电位与正电源持平，输出高电平，对应逻辑 1。

在实际工程应用中，与运算与与门电路的应用场景极为广泛，核心用于实现数字系统中的信号选通、地址译码、使能控制、逻辑权限判断等功能。例如在单片机系统中，我们可以通过与门电路，仅当“芯片使能信号有效”和“读写控制信号有效”两个条件同时满足时，才允许芯片执行数据读写操作，这正是与逻辑最典型的工程落地应用。

## 二、或运算（逻辑加运算）

### 1. 核心定义

或逻辑是与“与逻辑”并列的另一类基础因果逻辑，我们同样可以通过直观的电路案例理解其本质如图 1-1-4 所示，：在一个由直流电源、两个相互并联的开关 A、开关 B，以及一个负载灯泡$Y_2$共同组成的闭合回路中，只要开关 A 闭合或者开关 B 闭合，甚至两个开关同时闭合，整个回路就会处于导通状态，灯泡$Y_2$就会被点亮；只有当开关 A 和开关 B 全部都处于断开状态时，整个回路才会断路，灯泡$Y_2$才会熄灭。

图 1-1-4　或逻辑电路

从这个案例中，我们可以提炼出或逻辑的通用核心定义：在决定某一事件结果的多个前置条件中，只要有至少一个条件得到满足，该事件的结果就会发生；只有当所有条件全部都不满足时，事件结果才不会发生。这种“任一条件满足即可触发结果”的因果逻辑关系，就被称为或逻辑，与之对应的数学运算，就是或运算，也被称作逻辑加运算。

### 2. 逻辑表达式

或运算的通用逻辑表达式为：

$$Y = A + B$$

在该表达式中，A、B 为输入逻辑变量，代表决定事件结果的两个前置条件；Y 为输出逻辑变量，代表事件的最终结果；符号“+”为或运算的运算符号，读作“或”，也可读作“逻辑加”。需要特别强调的是，这里的“+”仅代表逻辑或的运算关系，与普通初等代数中的加法运算存在本质区别，不具备数值累加的含义。

当逻辑事件存在多个输入条件时，或运算可以直接扩展至多变量形式，n 个输入变量的或运算表达式为：

$$Y = A + B + C + \ldots + N$$

3. 或运算的真值表

对于双输入变量的或运算，同样存在$2^2 = 4$ 种输入取值组合，其完整真值表如下表所示：

表 1-1-2 或逻辑真值表

| 输入逻辑变量 A | 输入逻辑变量 B | 输出逻辑变量 Y | 对应逻辑含义 |
|---|---|---|---|
| 0 | 0 | 0 | 两个条件均不满足，事件不发生 |
| 0 | 1 | 1 | 条件 A 不满足、条件 B 满足，事件发生 |
| 1 | 0 | 1 | 条件 A 满足、条件 B 不满足，事件发生 |
| 1 | 1 | 1 | 两个条件均满足，事件发生 |

4. 核心运算规则

从上述真值表中，我们可以提炼出或运算的通用核心运算规则，也被称作或运算口诀：有 1 出 1，全 0 出 0。

该规则可拆解为两层核心含义：一是当所有输入逻辑变量中，存在任意一个变量取值为逻辑 1 时，无论其他输入变量的取值是逻辑 0 还是逻辑 1，最终的或运算输出结果必然为逻辑 1；二是只有当所有输入逻辑变量的取值，全部为逻辑 0 时，最终的或运算输出结果才会为逻辑 0。

基于该核心规则，我们可以直接推导出或运算的基础运算公式：

$$0 + 0 = 0、0 + 1 = 1、1 + 0 = 1、1 + 1 = 1$$

这里的 $1 + 1 = 1$，其逻辑含义为“两个满足事件触发的条件同时存在，事件依然会发生”，而非数值上的 1 加 1 等于 2，这也是逻辑加运算与普通代数加法最核心的区别，是逻辑代数初学者最容易混淆的知识点。该规则同样适用于多变量或运算，例如四变量或运算中，$0 + 1 + 0 + 0 = 1$，仅当 $0 + 0 + 0 + 0 = 0$，其余所有包含逻辑 1 的输入组合，输出结果均为逻辑 1。

5. 或门的电路实现与工程应用

实现或运算的门电路称为或门，二输入或门的标准逻辑符号见图 1-1-1，符号的输入端对应逻辑变量 A、B，输出端对应运算结果$Y_2$。

二输入或门同样可通过分立元件搭建，核心由两个硅二极管和一个限流电阻构成，电路结构与与门呈对称设计。两个二极管的阴极共接，经限流电阻接至电源地，两个二极管的阳极分别作为电路的两个信号输入端，阴极公共端作为电路的信号输出端。如图 1-1-5 所示。其工作

过程完全契合或运算的核心规则：当任意一个输入端接入高电平时，对应的二极管获得正向偏置而导通，输出端电位被钳位在高电平，对应逻辑 1；只有当两个输入端全部接入低电平时，两个二极管均处于反向截止状态，限流电阻无电流通过，输出端电位与地持平，输出低电平，对应逻辑 0。

图 1-1-5　二极管或门电路

在实际工程应用中，或运算与或门电路主要用于实现数字系统中的多路信号合并、报警触发、故障检测、多条件逻辑判断等功能。例如在工业安防系统中，我们可以通过或门电路，实现“烟雾传感器触发”“红外入侵传感器触发”“门禁异常传感器触发”任一条件满足时，立即触发报警装置，这正是或逻辑最典型的工程应用场景。

## 三、非运算（逻辑反运算/取反运算）

### 1. 核心定义

非逻辑是三种基本逻辑运算中，唯一的单变量运算，其核心是“互为否定”的因果关系。我们依然可以通过电路案例理解其本质，如图 1-1-6 所示：在一个由直流电源、一个常闭型开关 A、一个负载灯泡$Y_3$共同组成的闭合回路中，当开关 A 处于断开状态（条件不满足）时，回路保持导通，灯泡$Y_3$点亮；当开关 A 闭合（条件满足）时，回路被开关短路，灯泡$Y_3$立即熄灭。

图 1-1-6　非逻辑电路

从这个案例中，我们可以提炼出非逻辑的通用核心定义：事件的最终结果，与决定事件的前置条件呈完全相反的状态；条件满足时，事件结果不发生；条件不满足时，事件结果必然发生。这种“结果与条件完全互反”的因果逻辑关系，就被称为非逻辑，与之对应的数学运算，就是非运算，也被称作逻辑反运算、取反运算。

2. 逻辑表达式

非运算的通用逻辑表达式为：

$$Y = \overline{A}$$

在该表达式中，A 为输入逻辑变量，代表决定事件的前置条件；Y 为输出逻辑变量，代表事件的最终结果；变量 A 上方的横线“—”被称作非号，是取反运算的专用符号，该表达式读作“Y 等于 A 非”，也可读作“Y 等于 A 反”。

与与运算、或运算不同，非运算属于单目运算，仅能作用于单个逻辑变量，或单个完整的逻辑表达式，无法同时作用于多个独立的逻辑变量。若需要对多个变量分别取反，需要为每个变量单独添加非号，例如$\overline{A} \cdot \overline{B}$、$\overline{A} + \overline{B}$。

3. 真值表

非运算仅存在一个输入逻辑变量，因此仅有$2^1 = 2$ 种输入取值组合，其完整真值表如下表所示：

表 1-1-3　或逻辑真值表

| 输入逻辑变量 A | 输出逻辑变量 | 对应逻辑含义 |
|---|---|---|
| 0 | 1 | 条件不满足，事件发生 |
| 1 | 0 | 条件满足，事件不发生 |

4. 运算规则与定律

从上述真值表中，我们可以提炼出非运算的通用核心运算规则，也被称作非运算口诀：进 0 出 1，进 1 出 0。

该规则的核心含义为：非运算的输出结果，永远与输入变量的取值呈完全相反的互斥状态，不存在任何中间状态，这也是数字逻辑二值性最直观的体现。基于该规则，我们可以直接推导出非运算的基础运算公式：

$$\overline{0} = 1$$

$$\overline{1} = 0$$

在非运算的基础上，我们可以推导出逻辑代数中最基础的核心定律之一——还原律，也被称作双重否定律。其核心内容为：对一个逻辑变量连续进行两次非运算，最终的运算结果等于该变量本身，公式表达为：

$$\overline{\overline{A}} = A$$

该定律是逻辑代数化简、逻辑表达式变换中最常用的基础规则，我们可以进一步扩展其规律：对一个逻辑变量进行奇数次取反运算，最终结果等于原变量的取反；对一个逻辑变量进行偶数次取反运算，最终结果等于原变量本身。例如$\overline{\overline{\overline{A}}} = \overline{A}$、$\overline{\overline{\overline{A}}} = A$。

5.非门的电路实现与工程应用

实现非运算的门电路称为非门，也被普遍称作反相器，其标准逻辑符号见图 1-1-1，符号输出端的小圆圈代表取反运算，是反相器的核心标识。

非门的分立元件电路核心由一个 NPN 型三极管、两个限流电阻构成，三极管的基极经限流电阻接信号输入端，集电极经限流电阻接+5V 正电源，发射极直接接电源地，集电极作为电路的信号输出端。其工作过程完全契合非运算的核心规则：当输入端接入低电平时，三极管的发射结无正向偏置，三极管处于截止状态，集电极无电流通过，输出端电位与正电源持平，输出高电平，对应逻辑 1；当输入端接入高电平时，三极管的发射结获得正向偏置，三极管处于饱和导通状态，集电极与发射极之间的压降趋近于 0，输出端电位被钳位在低电平，对应逻辑 0。

图 1-1-7　非门电路

在实际工程应用中，非运算与非门电路主要用于实现数字系统中的信号相位翻转、电平转换、逻辑状态取反、时序电路中的时钟反相、总线驱动等功能。例如在单片机控制系统中，我们可以通过非门电路，将输出的高电平有效信号，转换为低电平有效信号，以匹配外围芯片的控制逻辑，这是非运算最常见的工程应用场景。

## 四、三种基本逻辑运算的优先级规则

在一个同时包含与运算、或运算、非运算的复杂逻辑表达式中，为了确保逻辑运算结果的唯一性，逻辑代数体系约定了固定的运算优先级规则，优先级从高到低依次为：

非运算>与运算>或运算

该优先级规则可进一步补充说明：

1.非运算的优先级最高，只要非号覆盖的变量或表达式，会优先执行取反运算；

2.与运算的优先级次之，优先级高于或运算，在无括号的情况下，先执行所有与运算，再执行或运算；

3.若逻辑表达式中包含括号，则括号内的运算优先级最高，优先执行括号内的所有逻辑运算，再按照优先级顺序执行括号外的运算；

4.同一优先级的运算，按照从左到右的顺序依次执行。

我们可以通过两个典型案例，清晰理解该优先级规则的应用：

1.逻辑表达式$Y = A + B \cdot C$：无括号，先执行优先级更高的 B 与 C 的与运算，再将运算结果与 A 执行或运算；

2.逻辑表达式$Y = \overline{A + B} \cdot C$：先执行括号内 A 与 B 的或运算，再对运算结果执行非运算，最后将取反后的结果与 C 执行与运算。

## 五、常用复合逻辑运算

在实际的工程应用与逻辑电路设计中，仅依靠三种基本逻辑运算，虽然在理论上可以实现所有逻辑功能，但在电路复杂度、运算效率、芯片集成度等方面往往无法满足实际需求。因此，在三种基本逻辑运算的基础上，逻辑代数体系进一步定义了五种常用复合逻辑运算，分别为与非运算、或非运算、与或非运算、异或运算和同或运算。

这五种复合逻辑运算均由基本逻辑运算组合而成，且各自拥有独立的逻辑符号、运算规则与工程意义，是数字集成电路设计中最核心的运算单元。其中，与非运算和或非运算具有逻辑完备性——即仅使用与非门或仅使用或非门，就可以实现所有的基本逻辑运算与复合逻辑运算，这一特性使得它们成为数字集成电路中应用最广泛的逻辑门类型。

**1.与非运算（NAND 运算）**

（1）定义

与非运算是与运算和非运算的直接组合，其运算顺序为“先与后非”。我们可以通过逻辑定义直观理解：只有当决定某一事件结果的全部条件都同时满足时，事件结果才不发生；只要有任意一个条件不满足，事件结果就会发生。这种因果逻辑关系，就是与非逻辑。

从电路角度理解，与非逻辑相当于在与门的输出端直接串联一个非门（反相器），将与门的输出结果取反后输出。

（2）逻辑表达式

双输入变量的与非运算通用逻辑表达式为：

$$Y = \overline{A \cdot B}$$

同样可简写为：

$$Y = \overline{AB}$$

在该表达式中，先执行 A 与 B 的与运算，再对与运算的结果执行非运算，非号覆盖了整个与运算表达式，这是运算优先级的直观体现。

与非运算可以直接扩展至多变量形式，n 个输入变量的与非运算表达式为：

$$Y = \overline{A \cdot B \cdot C \cdot \ldots \cdot N} = \overline{ABC \ldots N}$$

（3）真值表

对于双输入变量的与非运算，共有 4 种输入取值组合，其完整真值表如表 1-4 所示：

（4）核心运算规则

从上述真值表中，我们可以提炼出与非运算的通用核心运算规则，也被称作与非运算口诀：全 1 出 0，有 0 出 1。

表 1-1-4　与非逻辑运算真值表

| 输入变量 A | 输入变量 B | 中间与运算结果 AB | 输出逻辑变量$Y=\overline{AB}$ | 对应逻辑含义 |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 条件不全满足，事件发生 |
| 0 | 1 | 0 | 1 | 条件不全满足，事件发生 |
| 1 | 0 | 0 | 1 | 条件不全满足，事件发生 |
| 1 | 1 | 1 | 0 | 条件全满足，事件不发生 |

该规则是与运算规则的直接取反，可拆解为两层核心含义：一是只有当所有输入逻辑变量的取值全部为逻辑 1 时，最终的与非运算输出结果才会为逻辑 0；二是只要所有输入逻辑变量中存在任意一个变量取值为逻辑 0，无论其他输入变量的取值是逻辑 0 还是逻辑 1，最终的与非运算输出结果必然为逻辑 1。

基于该核心规则，我们可以直接推导出双输入与非运算的基础运算公式：

$$\overline{0\cdot 0}=\overline{0}=1$$

$$\overline{0\cdot 1}=\overline{0}=1$$

$$\overline{1\cdot 0}=\overline{0}=1$$

$$\overline{1\cdot 1}=\overline{1}=0$$

（5）与非门工程应用

实现与非运算的门电路称为与非门，二输入与非门的标准逻辑符号见图 1-1-1，符号输出端的小圆圈代表取反运算，对应“先与后非”的运算顺序。使用与非门这一种逻辑门，就可以通过不同的组合方式，实现与运算、或运算、非运算这三种基本逻辑运算，进而实现任意复杂的逻辑功能。这一特性使得与非门成为数字集成电路中设计最灵活、应用最广泛的逻辑门类型。

我们可以通过简单的公式变换，直观理解与非运算的逻辑完备性：

- 用与非门实现非运算：将与非门的所有输入端短接，输入同一个逻辑变量 A，即可实现非运算。公式推导为：$Y=\overline{A\cdot A}=\overline{A}$。
- 用与非门实现与运算：在与非门的输出端再串联一个与非门，并将第二个与非门的所有输入端短接，即可实现与运算。公式推导为：$Y=\overline{\overline{AB}\cdot\overline{AB}}=\overline{\overline{AB}}=AB$。

- 用与非门实现或运算：利用摩根定律，可以将或运算转换为与非运算的组合。公式推导为：$Y = A + B = \overline{\overline{A + B}} = \overline{\overline{A} \cdot \overline{B}}$，即通过三个与非门即可实现或运算。

在实际工程应用中，与非门电路主要用于实现数字系统中的逻辑函数化简、通用逻辑阵列、可编程逻辑器件、存储器地址译码等核心功能。由于其逻辑完备性，通用逻辑芯片中，与非门芯片的种类与数量也是最多的，广泛应用于各类数字电路的实验与设计中。

**2.或非运算（NOR 运算）**

（1）定义

或非运算是或运算和非运算的直接组合，其运算顺序为“先或后非”。我们可以通过逻辑定义直观理解：只有当决定某一事件结果的全部条件都不满足时，事件结果才会发生；只要有任意一个条件满足，事件结果就不会发生。这种因果逻辑关系，就是或非逻辑。

从电路角度理解，或非逻辑相当于在或门的输出端直接串联一个非门（反相器），将或门的输出结果取反后输出。

（2）逻辑表达式

双输入变量的或非运算通用逻辑表达式为：

$$Y = \overline{A + B}$$

在该表达式中，先执行 A 与 B 的或运算，再对或运算的结果执行非运算，非号覆盖了整个或运算表达式。

或非运算可以直接扩展至多变量形式，n 个输入变量的或非运算表达式为：

$$Y = \overline{A + B + C + \ldots + N}$$

（3）真值表

对于双输入变量的或非运算，共有 4 种输入取值组合，其完整真值表如下表所示：

表 1-1-5　或非逻辑运算真值表

| 输入逻辑变量 A | 输入逻辑变量 B | 中间或运算结果 A+B | 输出逻辑变量 Y | 对应逻辑含义 |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 条件全不满足，事件发生 |
| 0 | 1 | 1 | 0 | 条件有满足，事件不发生 |
| 1 | 0 | 1 | 0 | 条件有满足，事件不发生 |
| 1 | 1 | 1 | 0 | 条件有满足，事件不发生 |

（4）核心运算规则

从上述真值表中，我们可以提炼出或非运算的通用核心运算规则，也被称作或非运算口诀：全 0 出 1，有 1 出 0。

该规则是或运算规则的直接取反，可拆解为两层核心含义：一是只有当所有输入逻辑变量的取值全部为逻辑 0 时，最终的或非运算输出结果才会为逻辑 1；二是只要所有输入逻辑变量中存在任意一个变量取值为逻辑 1，无论其他输入变量的取值是逻辑 0 还是逻辑 1，最终的或非运算输出结果必然为逻辑 0。

基于该核心规则，我们可以直接推导出双输入或非运算的基础运算公式：

$$\overline{0+0}=\overline{0}=1$$

$$\overline{0+1}=\overline{1}=0$$

$$\overline{1+0}=\overline{1}=0$$

$$\overline{1+1}=\overline{1}=0$$

（5）或非门的工程应用

实现或非运算的门电路称为或非门，二输入或非门的标准逻辑符号见图 1-1-1，符号输出端的小圆圈代表取反运算，对应“先或后非”的运算顺序。

使用或非门这一种逻辑门，就可以通过不同的组合方式，实现与运算、或运算、非运算这三种基本逻辑运算，进而实现任意复杂的逻辑功能。这一特性使得或非门与与非门并列，成为数字集成电路中最重要的两种逻辑门类型。

我们同样可以通过简单的公式变换，直观理解或非运算的逻辑完备性：

- 用或非门实现非运算：将或非门的所有输入端短接，输入同一个逻辑变量 A，即可实现非运算。公式推导为：$Y=\overline{A+A}=\overline{A}$。
- 用或非门实现或运算：在或非门的输出端再串联一个或非门，并将第二个或非门的所有输入端短接，即可实现或运算。公式推导为：$Y=\overline{\overline{A+B}+\overline{A+B}}=\overline{\overline{A+B}}=A+B$。
- 用或非门实现与运算：利用摩根定律，可以将与运算转换为或非运算的组合。公式推导为：$Y=AB=\overline{\overline{AB}}=\overline{\overline{A}+\overline{B}}$，即通过三个或非门即可实现与运算。

在实际工程应用中，或非门电路同样广泛用于实现数字系统中的逻辑函数化简、通用逻辑阵列、可编程逻辑器件等核心功能。在基于 CMOS 工艺的专用集成电路设计中，或非门的电路结构往往比与非门更简单、更节省芯片面积，因此在这些场景中，或非门的应用频率甚至会高于与非门。

**3.与或非运算（AND-OR-NOT 运算）**

（1）定义

与或非运算是与运算、或运算、非运算三种基本逻辑运算的组合，其运算顺序为“先与、后或、再非”。我们可以通过逻辑定义直观理解：将决定事件结果的多个前置条件分为若干组，

只有当每一组内的所有条件都同时满足时，事件结果才不发生；只要有任意一组内的条件没有全部满足，事件结果就会发生。这种因果逻辑关系，就是与或非逻辑。

从电路角度理解，与或非逻辑相当于先通过多个与门对各组变量分别进行与运算，再通过一个或门将所有与运算的结果进行或运算，最后在或门的输出端串联一个非门，将或运算的结果取反后输出。

（2）逻辑表达式

最典型的与或非运算包含两组与运算，每组与运算包含两个输入变量，其通用逻辑表达式为：

$$Y = \overline{AB + CD}$$

在该表达式中，运算顺序严格遵循优先级规则：先执行 A 与 B 的与运算、C 与 D 的与运算，再执行两个与运算结果的或运算，最后对或运算的结果执行非运算。

与或非运算可以根据实际需求扩展为更多组与运算，每组与运算也可以包含更多的输入变量，其通用扩展表达式为：

$$Y = \overline{AB + CD + EF + \ldots}$$

（3）真值表

对于包含两组二输入与运算的与或非运算，共有$2^4 = 16$ 种输入取值组合，其完整真值表如下表所示：

表 1-1-6　与或非逻辑运算真值表

| 输入 A | 输入 B | 输入 C | 输入 D | 中间与项 AB | 中间与项 CD | 中间或项 AB+CD | 输出 Y=AB+CD |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |

（4）核心运算规则

从上述真值表中，我们可以提炼出与或非运算的通用核心运算规则：全与项为 1 出 0，有与项为 0 出 1。

该规则可拆解为两层核心含义：

1.只有当表达式中所有的与项（即每一组与运算的结果）取值全部为逻辑 1 时，最终的与或非运算输出结果才会为逻辑 0；

2.只要表达式中存在任意一个与项取值为逻辑 0，无论其他与项的取值是逻辑 0 还是逻辑 1，最终的与或非运算输出结果必然为逻辑 1。

（5）与或非门的工程应用

实现与或非运算的门电路称为与或非门，二组二输入与或非门的标准逻辑符号见图 1-1-1，符号内部包含两组与逻辑输入，经或逻辑后输出端带取反标识，完整对应“先与、后或、再非”的运算顺序。

与或非运算虽然不具备逻辑完备性，但它是逻辑函数“与或表达式”的直接取反形式，而与或表达式是逻辑代数中最常用、最直观的逻辑函数表达形式之一。因此，在数字系统设计中，与或非运算主要用于直接实现基于与或表达式的逻辑函数，无需进行额外的逻辑变换，从而简化了设计流程。在实际工程应用中，与或非门电路常用于实现数据选择器、译码器、编码器、算术逻辑单元（ALU）等复杂数字逻辑模块的核心功能。

**4.异或运算（XOR 运算，Exclusive-OR）**

（1）定义

异或运算是一种特殊的二变量逻辑运算，其核心逻辑含义可以直观理解为：当两个输入逻辑变量的取值相同时，输出结果为逻辑 0；当两个输入逻辑变量的取值不同时，输出结果为逻辑 1。这种“相同出 0，不同出 1”的因果逻辑关系，就是异或逻辑。

异或运算虽然可以通过基本逻辑运算组合实现（公式为$Y = A\overline{B} + \overline{A}B$），但由于其在数字系统中应用极为广泛，因此被定义为独立的复合逻辑运算，并拥有专属的运算符号与逻辑门。

（2）逻辑表达式

异或运算拥有专属的运算符号“⊕”，读作“异或”，其通用逻辑表达式为：

$$Y = A \oplus B$$

如前文所述，异或运算可以通过基本逻辑运算的组合来等价表示，这也是异或运算逻辑化简与电路实现的基础，其等价与或表达式为：

$$Y = A \oplus B = A\overline{B} + \overline{A}B$$

需要特别注意的是，异或运算仅适用于两个输入变量，虽然可以通过多个异或门级联实现多变量的异或逻辑，但多变量异或运算的逻辑含义与二变量异或运算存在本质区别，不能简单套用“相同出 0，不同出 1”的规则。

（3）真值表

对于双输入变量的异或运算，共有 4 种输入取值组合，其完整真值表如下表所示：

表 1-1-7　异或逻辑运算真值表

| 输入逻辑变量 A | 输入逻辑变量 B | 输出逻辑变量$Y = A \oplus B$ | 对应逻辑含义 |
|---|---|---|---|
| 0 | 0 | 0 | 输入相同，输出 0 |
| 0 | 1 | 1 | 输入不同，输出 1 |
| 1 | 0 | 1 | 输入不同，输出 1 |
| 1 | 1 | 0 | 输入相同，输出 0 |

（4）运算规则与常用公式

从上述真值表中，我们可以提炼出异或运算的通用核心运算规则，也被称作异或运算口诀：相同出 0，不同出 1。

基于该核心规则，我们可以直接推导出异或运算的基础运算公式：

$$0 \oplus 0 = 0、0 \oplus 1 = 1、1 \oplus 0 = 1、1 \oplus 1 = 0$$

除了基础运算公式外，异或运算还拥有一系列在逻辑化简、逻辑变换中极为常用的运算公式，这些公式是数字逻辑电路设计的重要工具，需要重点掌握：

(1) 变量与自身异或：$A \oplus A = 0$

逻辑含义：任何逻辑变量与自身异或，由于取值必然相同，因此结果恒为逻辑 0。

(2) 变量与 0 异或：$A \oplus 0 = A$

逻辑含义：任何逻辑变量与逻辑 0 异或，结果等于变量本身。

(3) 变量与 1 异或：$A \oplus 1 = \overline{A}$

逻辑含义：任何逻辑变量与逻辑 1 异或，结果等于变量的取反。这一公式使得异或门可以作为可控反相器使用——当控制端为 1 时，输出等于输入的取反；当控制端为 0 时，输出等于输入本身。

(4) 交换律：$A \oplus B = B \oplus A$

逻辑含义：异或运算的两个输入变量交换顺序，运算结果不变。

(5) 结合律：$(A \oplus) \oplus C = A \oplus (B \oplus C)$

逻辑含义：在多变量异或运算中，运算顺序不影响最终结果。基于结合律，我们可以定义多变量异或运算：$A \oplus B \oplus C \oplus \ldots$，其运算规则为：当输入变量中取值为 1 的变量个数为奇数时，输出结果为 1；当取值为 1 的变量个数为偶数时，输出结果为 0。这一规则是奇偶校验电路的核心逻辑基础。

（5）异或门的工程应用

实现异或运算的门电路称为异或门，二输入异或门的标准逻辑符号见图 1-1-1，符号内部标注有“=1”的功能标识，对应“输入取值不同时输出 1”的核心规则。

异或运算在数字系统中拥有不可替代的核心工程价值，是算术运算电路、数据传输校验电路、密码学加密电路等核心模块的逻辑基础。在实际工程应用中，异或运算与异或门电路的典型应用场景包括：

- 二进制加法运算：在二进制半加器与全加器电路中，异或门用于实现两个一位二进制数的“本位和”运算——不考虑低位进位时，两个二进制数相加的本位和，恰好等于这两个数的异或运算结果。这是异或运算最重要的应用场景，是所有算术逻辑单元（ALU）的核心基础。
- 奇偶校验电路：基于多变量异或运算的规则，异或门可以用于实现奇偶校验电路，用于检测数据传输过程中是否发生了误码。例如，在发送端通过异或门计算数据的奇偶校验位并一同发送，在接收端再次通过异或门计算校验位并比对，若不一致则说明数据传输出错。
- 可控反相器：基于“变量与 1 异或等于取反”的规则，异或门可以作为可控反相器使用，通过控制端的电平，选择将输入信号直接输出或取反后输出。这一功能广泛用于数据选择器、补码运算电路等模块中。
- 序列密码加密：在现代密码学中，异或运算是序列密码（流密码）的核心加密运算——将明文数据与密钥序列进行逐位异或运算，即可得到密文；解密时只需将密文与相同的密钥序列再次进行逐位异或运算，即可还原出明文。

**5.同或运算（XNOR 运算，Exclusive-NOR）**

（1）定义

同或运算是异或运算的直接取反，因此也被称作“异或非运算”，其核心逻辑含义可以直观理解为：当两个输入逻辑变量的取值相同时，输出结果为逻辑 1；当两个输入逻辑变量的取值不同时，输出结果为逻辑 0。这种“相同出 1，不同出 0”的因果逻辑关系，就是同或逻辑。

与异或运算类似，同或运算也可以通过基本逻辑运算组合实现（公式为$Y = AB + \overline{AB}$），同样由于其广泛的应用价值，被定义为独立的复合逻辑运算，拥有专属的运算符号与逻辑门。

（2）逻辑表达式

同或运算拥有专属的运算符号“$\odot$”，读作“同或”，也可读作“异或非”，其通用逻辑表达式为：

$$Y = A \odot B$$

$$Y = A \odot B = \overline{A \oplus B} = AB + \overline{AB}$$

与异或运算相同，同或运算也仅适用于两个输入变量，多变量同或运算需要通过级联实现，且逻辑含义与二变量同或运算存在区别。

（3）真值表

对于双输入变量的同或运算，共有 4 种输入取值组合，其完整真值表如下表所示：

表 1-1-8 同或逻辑运算真值表

| 输入逻辑变量 A | 输入逻辑变量 B | 输出逻辑变量$Y = A \odot B$ | 对应逻辑含义 |
|---|---|---|---|
| 0 | 0 | 1 | 输入相同，输出 1 |
| 0 | 1 | 0 | 输入不同，输出 0 |
| 1 | 0 | 0 | 输入不同，输出 0 |
| 1 | 1 | 1 | 输入相同，输出 1 |

（4）运算规则与常用公式

从上述真值表中，我们可以提炼出同或运算的通用核心运算规则，也被称作同或运算口诀：相同出 1，不同出 0。

基于该核心规则，我们可以直接推导出同或运算的基础运算公式：

$$0 \odot 0 = 1、0 \odot 1 = 0、1 \odot 0 = 0、1 \odot 1 = 1$$

与异或运算类似，同或运算也拥有一系列常用的运算公式，这些公式与异或运算的公式具有明显的对称性，在逻辑化简与变换中同样具有重要价值：

(1) 变量与自身同或：$A \odot A = 1$

逻辑含义：任何逻辑变量与自身同或，由于取值必然相同，因此结果恒为逻辑 1。

(2) 变量与 0 同或：$A \odot 0 = \overline{A}$

逻辑含义：任何逻辑变量与逻辑 0 同或，结果等于变量的取反。

(3) 变量与 1 同或：$A \odot 1 = A$

逻辑含义：任何逻辑变量与逻辑 1 同或，结果等于变量本身。

(4) 交换律：$A \odot B = B \odot A$

逻辑含义：同或运算的两个输入变量交换顺序，运算结果不变。

(5) 结合律：$(A \odot) \odot C = A \odot (B \odot C)$

逻辑含义：在多变量同或运算中，运算顺序不影响最终结果。

(6) 与异或运算的互反关系：$A \odot B = \overline{A \oplus B}$，且$A \oplus B = \overline{A \odot B}$

逻辑含义：同或运算与异或运算互为反函数，对其中一个运算的结果取反，即可得到另一个运算的结果。

（5）同或门的工程应用

实现同或运算的门电路称为同或门，也叫异或非门，二输入同或门的标准逻辑符号见图 1-1-1，符号输出端的小圆圈代表取反运算，对应异或门输出取反的核心逻辑。

同或运算作为异或运算的互补运算，在数字系统中同样拥有广泛的应用场景，其核心价值与异或运算具有对称性。在实际工程应用中，同或运算与同或门电路的典型应用场景包括：

- 数值比较器：在一位二进制数值比较器电路中，同或门用于判断两个一位二进制数是否相等——当两个数相同时，同或运算输出 1；当两个数不同时，同或运算输出 0。这是同或运算最典型的应用场景，是多位数值比较器的核心基础。
- 奇偶校验电路（互补型）：与异或运算类似，同或运算也可以用于实现奇偶校验电路，且二者的校验结果具有互补性。例如，当异或门用于实现奇校验时，同或门即可用于实现偶校验，反之亦然。
- 数据一致性检测：在数字系统中，同或门可以用于检测两组并行数据是否完全一致——将两组数据的对应位分别输入多个同或门，只有当所有同或门的输出都为 1 时，才说明两组数据完全一致；若有任意一个同或门输出为 0，则说明两组数据存在差异。

### 1.1.2 逻辑代数的基本公式与定理

在 1.1.1 节中，我们系统介绍了逻辑代数的基本逻辑运算与复合逻辑运算，明确了逻辑变量与逻辑运算的核心规则。而逻辑代数的基本公式、常用公式与三大基本定理，是基于基本逻辑运算的客观规律推导而来的布尔恒等式与通用运算规则，是逻辑函数化简、逻辑表达式变换、数字逻辑电路分析与设计的核心数学工具。所有逻辑恒等式的成立，都可以通过真值表进行最直观的验证——若等式两边的逻辑表达式，在输入变量的所有取值组合下，输出结果均完全一致，则该恒等式成立。

**一、逻辑代数的基本公式**

逻辑代数的基本公式共分为 9 组，完整覆盖了常量与常量、变量与常量、变量与变量之间的所有核心运算规律，是整个逻辑代数体系的运算基础。其中与普通代数运算规则一致的公式，可直接沿用已有数学认知理解；而逻辑代数特有的公式，是初学者需要重点区分和掌握的核心内容，也是数字逻辑电路设计区别于模拟电路计算的关键。

**1.常量与变量的基础运算公式**

本组公式直接源于 1.1.1 节中三种基本逻辑运算的核心规则，描述了逻辑常量 0、1 与逻辑变量之间的固定运算关系，是所有逻辑公式推导的起点。

（1）与运算相关公式

与运算的核心规则为“有 0 出 0，全 1 出 1”，由此可推导出变量与常量的固定运算关系：

常量间运算：$0 \cdot 0 = 0$；$0 \cdot 1 = 0$；$1 \cdot 0 = 0$；$1 \cdot 1 = 1$

归零律：$A \cdot 0 = 0$

逻辑含义：无论逻辑变量 A 取值为 0 还是 1，与逻辑 0 进行与运算时，结果恒为 0。对应电路中，与门的任意一个输入端接低电平，输出端必然固定为低电平，与其他输入信号无关。

自等律：$A \cdot 1 = A$

逻辑含义：逻辑变量 A 与逻辑 1 进行与运算时，结果恒等于变量 A 本身。对应电路中，与门的任意一个输入端接高电平，输出端的逻辑状态完全由其他输入信号决定，不会改变原有逻辑关系。

（2）或运算相关公式

或运算的核心规则为“有 1 出 1，全 0 出 0”，由此可推导出变量与常量的固定运算关系：

常量间运算：$0+0=0$；$0+1=1$；$1+0=1$；$1+1=1$

自等律：$A+0=A$

逻辑含义：逻辑变量 A 与逻辑 0 进行或运算时，结果恒等于变量 A 本身。对应电路中，或门的任意一个输入端接低电平，输出端的逻辑状态完全由其他输入信号决定，不会改变原有逻辑关系。

置 1 律：$A+1=1$

逻辑含义：无论逻辑变量 A 取值为 0 还是 1，与逻辑 1 进行或运算时，结果恒为 1。对应电路中，或门的任意一个输入端接高电平，输出端必然固定为高电平，与其他输入信号无关。

（3）非运算相关公式与互补律

非运算的核心规则为“进 0 出 1，进 1 出 0”，由此可推导出逻辑代数中最核心的互补律，是逻辑化简中应用最频繁的基础公式之一：

常量取反：$\overline{0}=1$；$\overline{1}=0$

互补律：$A\cdot\overline{A}=0$；$A+\overline{A}=1$

逻辑含义：逻辑变量 A 与其反变量$\overline{A}$，取值必然呈互斥状态——一个为 0 时另一个必为 1，因此二者相与的结果恒为 0，二者相或的结果恒为 1。这是逻辑代数特有的定律，普通代数中不存在“一个数与其相反数相乘恒为 0、相加恒为 1”的规律，也是逻辑函数化简、互补逻辑电路设计的核心基础。

**2.与普通代数共有的通用运算定律**

本组公式的运算规律与普通初等代数完全一致，可直接沿用通用的运算逻辑理解，主要用于调整逻辑表达式的运算顺序、展开与合并逻辑项，为逻辑化简提供基础变换规则。

（1）交换律

交换律描述了逻辑运算的结果与输入变量的顺序无关，仅由变量的取值决定，适用于与运算、或运算、异或运算、同或运算：

与运算交换律：$A\cdot B=B\cdot A$

或运算交换律：$A+B=B+A$

逻辑含义：在与运算和或运算中，交换两个输入变量的位置，最终的运算结果保持不变。对应电路中，与门、或门的各个输入端之间没有顺序差异，互换输入信号的接入位置，不会改变门电路的输出逻辑。

（2）结合律

结合律描述了多变量连续运算时，运算顺序的调整不会改变最终结果，适用于多变量的与运算和或运算：

与运算结合律：$(A \cdot B) \cdot C = A \cdot (B \cdot C)$

或运算结合律：$(A + B) + C = A + (B + C)$

逻辑含义：在多变量连续与运算、连续或运算中，可任意调整运算的先后顺序，最终结果保持不变。该定律使得多变量逻辑运算可根据化简需求灵活分组，对应电路中，多个二输入与门级联实现多输入与逻辑时，级联顺序不会改变最终的输出逻辑。

（3）分配律

分配律描述了与运算和或运算之间的嵌套运算规则，逻辑代数中包含两组分配律，其中第一组与普通代数的乘法分配律完全一致，第二组为逻辑代数特有，是逻辑表达式因式分解、展开变换的核心规则。

第一分配律（乘对加的分配）：$A \cdot (B + C) = A \cdot B + A \cdot C$

逻辑含义：一个变量与一组或运算的结果相与，等价于该变量分别与括号内的每个变量相与，再将所有与运算的结果相或。该定律与普通代数的乘法分配律逻辑完全一致，主要用于逻辑表达式的展开运算。

第二分配律（加对乘的分配）：$A + B \cdot C = (A + B) \cdot (A + C)$

逻辑含义：一个变量与一组与运算的结果相或，等价于该变量分别与括号内的每个变量相或，再将所有或运算的结果相与。这是逻辑代数特有的运算定律，普通初等代数中不存在该运算规律，需通过公式推导验证其成立性：

$$
\begin{aligned}
(A + B) \cdot (A + C) &= A \cdot A + A \cdot C + B \cdot A + B \cdot C \\
&= A + A \cdot C + A \cdot B + B \cdot C \\
&= A \cdot (1 + C + B) + B \cdot C \\
&= A \cdot 1 + B \cdot C \\
&= A + B \cdot C
\end{aligned}
$$

该定律主要用于逻辑表达式的因式分解，是将与或表达式转换为或与表达式的核心工具，对应电路中可实现与门、或门结构的灵活转换。

**3.逻辑代数特有的核心运算定律**

本组公式是逻辑代数区别于普通初等代数的核心特征，完全基于二值逻辑的基本规则推导而来，是逻辑函数化简、逻辑表达式变换中最核心的基础定律。

（1）重叠律

重叠律描述了逻辑变量与自身进行运算的固定结果，源于逻辑变量仅存在 0 和 1 两种取值的二值特性：

与运算重叠律：$A \cdot A = A$

或运算重叠律：$A + A = A$

逻辑含义：任何逻辑变量与自身进行与运算、或运算，结果恒等于变量本身。这与普通代数完全不同——普通代数中$a \cdot a = a^2$、$a + a = 2a$，而逻辑代数中不存在数值的累加与幂次运算，仅表征逻辑状态。该定律主要用于合并逻辑表达式中重复的与项、重复的变量，简化表达式结构，例如$AB + AB = AB$、$ABC \cdot BC = ABC$。

（2）还原律（双重否定律）

还原律源于非运算的互反特性，在 1.1.1 节非运算部分已有初步介绍，是逻辑表达式取反、反演运算的核心基础：

还原律公式：$\overline{\overline{A}} = A$

逻辑含义：对一个逻辑变量连续进行两次非运算，最终结果等于原变量本身。可进一步扩展为通用规律：对逻辑变量进行奇数次取反，结果等于原变量的反；对逻辑变量进行偶数次取反，结果等于原变量本身。该定律广泛应用于逻辑表达式的变换，例如利用与非门实现与运算、利用或非门实现或运算，核心就是通过还原律实现两次取反等价于原逻辑的效果。

（3）反演律（摩根定理）

反演律也被称作摩根定理，是由英国数学家奥古斯塔斯·德·摩根提出的核心定律，是逻辑代数中应用最广泛的定律之一，揭示了与运算和或运算之间的互反转换关系，也是 1.1.1 节中与非运算、或非运算逻辑完备性的核心理论基础，同时是后续反演定理的特例形式。

基本形式（二变量）

与非转换公式：$\overline{A \cdot B} = \overline{A} + \overline{B}$

或非转换公式：$\overline{A + B} = \overline{A} \cdot \overline{B}$

扩展形式（多变量）

多变量与非转换：$\overline{A \cdot B \cdot C \cdot \ldots \cdot N} = \overline{A} + \overline{B} + \overline{C} + \ldots + \overline{N}$

多变量或非转换：$\overline{A + B + C + \ldots + N} = \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot \ldots \cdot \overline{N}$

核心含义与验证

反演律的核心逻辑可概括为两句话：与运算的反，等于各变量反的或运算；或运算的反，等于各变量反的与运算。该定律的成立性可通过真值表直接验证，以二变量与非转换公式$\overline{A \cdot B} = \overline{A} + \overline{B}$为例，其完整真值表如下：

表 1-1-9 真值表

| 输入变量 A | 输入变量 B | 左边表达式 $A\overline{B}$ | 右边表达式 $\overline{A}+\overline{B}$ |
|---|---|---|---|
| 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |

从真值表可清晰看到，在所有输入取值组合下，等式两边的输出结果完全一致，恒等式成立。

**二、逻辑代数的常用公式**

常用公式是由上述基本公式推导而来的实用恒等式，全部针对逻辑函数化简的高频场景设计，无需每次从基本公式开始推导，可直接用于消去冗余项、冗余因子、合并同类项，大幅简化逻辑化简的流程，降低数字电路的设计复杂度。所有常用公式均以与或表达式为基础形式，这也是逻辑代数中最常用的逻辑表达式形式。

1.吸收律 1（消项公式）

公式形式：$A + A \cdot B = A$

公式推导：基于分配律与置 1 律，$A + AB = A \cdot (1 + B) = A \cdot 1 = A$

核心含义：在与或逻辑表达式中，如果一个与项是另一个与项的全部因子，那么包含该因子的长与项是完全冗余的，可以直接从表达式中消去。

应用示例：

$A + ABCD = A$（与项 A 是 ABCD 的全部因子，ABCD 为冗余项，直接消去）

$AB + ABC + ABD = AB$（与项 AB 是 ABC、ABD 的全部因子，后两项为冗余项，直接消去）

工程意义：直接减少逻辑表达式中的与项数量，对应电路中可减少与门的使用数量，降低电路复杂度。

2.吸收律 2（消因子公式）

公式形式：$A + \overline{A} \cdot B = A + B$

公式推导：基于第二分配律与互补律，$A + \overline{A}B = \left(A + \overline{A}\right) \cdot (A + B) = 1 \cdot (A + B) = A + B$

核心含义：在与或逻辑表达式中，如果一个与项的反变量是另一个与项的一个因子，那么这个反变量因子是完全冗余的，可以直接从与项中消去。

应用示例：

$A + \overline{A}BC = A + BC$（反变量$\overline{A}$是 BC 项的因子，$\overline{A}$为冗余因子，直接消去）

$AB + \overline{AB}CD = AB + CD$（反变量$\overline{AB}$是 CD 项的因子，$\overline{AB}$为冗余因子，直接消去）

工程意义：减少单个与项中的变量数量，对应电路中可减少与门的输入端数量，简化门电路的接线设计。

3.合并律（并项公式）

$ABC + AB\overline{C} = AB$（互补变量为 C 和$\overline{C}$，合并后消去互补变量，保留公共因子 AB）

$A\overline{B}C + \overline{AB}C = \overline{B}C$（互补变量为 A 和$\overline{A}$，合并后消去互补变量，保留公共因子$\overline{B}C$）

公式形式：$A \cdot B + A \cdot \overline{B} = A$

公式推导：基于分配律与互补律，$AB + A\overline{B} = A \cdot \left(B + \overline{B}\right) = A \cdot 1 = A$

核心含义：在与或逻辑表达式中，如果两个与项除了一个互补的变量外，其余所有因子完全相同，那么这两个与项可以合并为一个与项，直接消去互补的变量。

应用示例：

$ABC + AB\overline{C} = AB$（互补变量为 C 和$\overline{C}$，合并后消去互补变量，保留公共因子 AB）

$A\overline{B}C + \overline{AB}C = \overline{B}C$（互补变量为 A 和$\overline{A}$，合并后消去互补变量，保留公共因子$\overline{B}C$）

工程意义：将两个与项合并为一个，同时减少变量数量，是后续卡诺图化简法的核心理论基础。

4.冗余律（消冗余项公式，共识定理）

基本公式形式：$A \cdot B + \overline{A} \cdot C + B \cdot C = A \cdot B + \overline{A} \cdot C$

扩展公式形式：$A \cdot B + \overline{A} \cdot C + B \cdot C \cdot D = A \cdot B + \overline{A} \cdot C$

公式推导：基于互补律与重叠律，$AB + \overline{A}C + BC = AB + \overline{A}C + BC\left(A + \overline{A}\right) = AB + \overline{A}C + ABC + \overline{A}BC = AB(1 + C) + \overline{A}C(1 + B) = AB + \overline{A}C$

核心含义：在与或逻辑表达式中，如果两个与项分别包含互补的变量 A 和$\overline{A}$，且这两个与项的其余所有因子共同组成了第三个与项，那么第三个与项是完全冗余的，可以直接从表达式中消去；即使第三个与项包含其他额外的公共因子，依然属于冗余项，可直接消去。

应用示例：

$AB + \overline{A}C + BCD = AB + \overline{A}C$（BC 是 AB 与$\overline{A}C$的公共因子组成的与项，BCD 为冗余项，直接消去）

$ABC + \overline{A}D + BCD = ABC + \overline{A}D$（BC 是 ABC 与$\overline{A}D$的公共因子组成的与项，BCD 为冗余项，直接消去）

工程意义：消去逻辑表达式中隐藏的冗余项，避免设计多余的门电路，在保证逻辑功能不变的前提下，最大限度简化电路结构。

5.异或与同或运算互反公式

本组公式源于 1.1.1 节中异或、同或运算的核心定义，由基本公式推导而来，是互补逻辑电路设计的核心工具：

基本公式：$\overline{A \oplus B} = A \odot B$；$\overline{A \odot B} = A \oplus B$

公式推导：基于反演律，$\overline{A \oplus B} = \overline{A\overline{B} + \overline{A}B} = \overline{A\overline{B}} \cdot \overline{\overline{A}B} = (\overline{A} + B)(A + \overline{B}) = AB + \overline{A}\overline{B} = A \odot B$

核心含义：异或运算与同或运算互为反函数，对异或运算的结果取反即可得到同或运算，反之亦然。

工程意义：可通过异或门串联非门实现同或门，也可通过同或门串联非门实现异或门，在门电路类型受限的场景下，可灵活实现两种逻辑功能的转换。

**三、逻辑代数的三大基本定理**

逻辑代数的三大基本定理分别为代入定理、反演定理、对偶定理，是基于基本公式推导而来的通用运算规则，是连接基础公式与工程应用的核心桥梁，可实现逻辑公式的无限扩展、反函数的快速求解、逻辑恒等式的简化验证，是本节必须掌握的核心教学内容。

**1.代入定理**

（1）核心定义

在任何一个成立的逻辑恒等式中，若将等式两边所有出现的某一个逻辑变量，都用同一个完整的逻辑函数来替代，替代后的等式依然成立，这一规则被称为代入定理。

（2）原理

逻辑代数的核心基础是二值逻辑，无论是单个逻辑变量，还是复杂的逻辑函数，其最终的取值只有逻辑 0 和逻辑 1 两种可能。因此，用一个逻辑函数替代原有的变量，本质上只是改变了变量的取值来源，并未改变二值逻辑的基本规则，原恒等式的成立性不会受到任何影响。

（3）应用示例与注意事项

基础应用示例：利用代入定理扩展摩根定理的多变量形式

已知二变量摩根定理$\overline{A \cdot B} = \overline{A} + \overline{B}$恒成立，将等式两边所有出现的变量 B，用逻辑函数$Y = C \cdot D$替代，根据代入定理，替代后的等式依然成立：

$$\overline{A \cdot (C \cdot D)} = \overline{A} + \overline{C \cdot D}$$

再对等式右侧的$\overline{C \cdot D}$应用二变量摩根定理，即可得到三变量摩根定理：

$$\overline{A \cdot C \cdot D} = \overline{A} + \overline{C} + \overline{D}$$

以此类推，可通过代入定理无限扩展到 n 变量的摩根定理，与前文给出的多变量扩展形式完全一致。

进阶应用示例：扩展吸收律公式

已知吸收律$A + AB = A$恒成立，将等式两边所有出现的变量 A，用逻辑函数$Y = C + D$替代，替代后的等式依然成立：

$$(C + D) + (C + D) \cdot B = C + D$$

该公式可直接用于多变量逻辑表达式的化简，无需重新推导。

严格遵守的注意事项：

1.必须对等式两边所有出现该变量的位置进行同步替代，不能只替代部分位置，否则会破坏等式的成立性；

2.替代的必须是一个完整的逻辑函数，替代后需通过括号保证原有的运算优先级不被改变，避免出现运算顺序错误。

**2.反演定理**

（1）核心定义

对于任意一个逻辑函数 Y，若将其表达式中所有的符号与变量按照以下三条规则同步变换：

- 运算符变换：将所有的“·”（与运算）换成“+”（或运算），所有的“+”（或运算）换成“·”（与运算）；
- 常量变换：将所有的常量 0 换成 1，所有的常量 1 换成 0；
- 变量变换：将所有的原变量换成反变量，所有的反变量换成原变量；

那么变换后得到的全新逻辑表达式，就是原函数 Y 的反函数$\overline{Y}$，这一规则被称为反演定理。

反演定理是摩根定理的通用扩展形式，摩根定理仅适用于简单逻辑表达式的取反，而反演定理可一步到位求解任意复杂逻辑函数的反函数，无需再通过摩根定理进行多步推导，大幅简化了反函数的求解过程。

（2）必须遵守的两大规则

反演定理的应用场景多为复杂逻辑表达式，初学者极易出现运算顺序错误，因此必须严格遵守以下两条规则，保证反函数求解的准确性：

- 运算优先级规则：变换后的表达式必须严格保持原函数的运算优先级，即“括号>非运算>与运算>或运算”的顺序不变。原表达式中先运算的部分，变换后依然要先运算，必要时必须添加括号保证运算顺序，禁止随意改变原有的运算逻辑。
- 非号处理规则：仅覆盖单个变量的短非号，在变量取反后直接去掉；覆盖多个变量、整个与项或完整表达式的长非号，仅对非号下方的内容按照规则进行变换，长非号本身必须完整保留，不得随意去掉。

（3）应用示例

示例 1：求解逻辑函数$Y = AB + \overline{A}C$的反函数$\overline{Y}$

按照反演定理规则逐步变换：

1) 运算符变换：·换+，+换·，得到$(A + B) \cdot (\overline{A} + C)$；
2) 变量变换：原变量换反变量，反变量换原变量，得到$(\overline{A} + \overline{B}) \cdot (A + \overline{C})$；
3) 无常量变换，保持运算优先级不变，最终得到反函数：

$$\overline{Y} = (\overline{A} + \overline{B}) \cdot (A + \overline{C})$$

可通过摩根定理验证：$\overline{Y}=\overline{AB+\overline{A}C}=\overline{AB}\cdot\overline{\overline{A}C}=\left(\overline{A}+\overline{B}\right)\left(A+\overline{C}\right)$，与反演定理求解结果完全一致。

示例 2：求解逻辑函数$Y=A+\overline{B+\overline{C}+\overline{D+\overline{E}}}$的反函数$\overline{Y}$

按照反演定理规则与非号处理规则逐步变换：

1) 运算符变换：+换·，·换+，原表达式无·运算，仅需变换+；
2) 常量变换：无常量，无需变换；
3) 变量变换：原变量换反变量，反变量换原变量；
4) 长非号处理：覆盖$B+\overline{C}+\overline{D+\overline{E}}$的长非号、覆盖$D+\overline{E}$的长非号均保留，仅变换非号内部内容；

最终得到反函数：

$$\overline{Y}=\overline{A}\cdot\overline{\overline{B}\cdot C\cdot\overline{\overline{D}\cdot E}}$$

**3.对偶定理**

（1）定义

对于任意一个逻辑函数 Y，若将其表达式中所有的符号按照以下规则同步变换：

- 运算符变换：将所有的“·”（与运算）换成“+”（或运算），所有的“+”（或运算）换成“·”（与运算）；
- 常量变换：将所有的常量 0 换成 1，所有的常量 1 换成 0；
- 变量保持不变：所有原变量、反变量均保持原样，不进行任何取反变换；

那么变换后得到的全新逻辑表达式，就是原函数 Y 的对偶式，记作$Y^{'}$（或$Y^{D}$）。

需要特别注意的是，对偶式的变换规则与反演定理的核心区别在于：对偶式仅变换运算符和常量，变量完全保持不变；而反演定理需要同时变换运算符、常量和变量，二者不可混淆。

若两个逻辑恒等式$Y_1=Y_2$成立，那么它们对应的对偶式$Y_1^{'}=Y_2^{'}$也一定成立，这一规则被称为对偶定理。

（2）应用

对偶定理最核心的作用，是将逻辑代数的基本公式与常用公式的记忆量减少一半。观察前文的基本公式可以发现，所有的公式都是成对出现的，每一对公式互为对偶式，只要记住其中一个，就可以通过对偶定理直接得到另一个，无需逐一推导和记忆。

示例：已知乘对加的分配律$A\cdot(B+C)=AB+AC$成立，其对偶式为 A+B·C=(A+B)(A+C)，根据对偶定理，该式必然成立，也就是前文的第二分配律；

已知吸收律$A+AB=A$成立，其对偶式为$A\cdot(A+B)=A$，根据对偶定理，该式必然成立，也就是与运算的吸收律；

已知常用公式$A+\overline{A}B=A+B$成立，其对偶式为$A\cdot\left(\overline{A}+B\right)=AB$，根据对偶定理，该式必然成立，可直接用于逻辑化简。

除此之外，对偶定理还可用于逻辑恒等式的简化验证：对于一个复杂的逻辑恒等式，若直接证明其成立难度较大，可转而证明其对偶式成立，只要对偶式成立，原恒等式必然成立，大幅简化了推导过程。

（3）注意事项

与反演定理一致，求解对偶式时必须严格保持原函数的运算优先级不变，原表达式中先运算的部分，变换后依然要先运算，必要时添加括号保证运算顺序，避免出现逻辑错误。

**四、逻辑恒等式的验证方法与综合应用**

1.逻辑恒等式的两大验证方法

所有逻辑代数的基本公式、常用公式与定理推导结果，均可通过两种核心方法验证其成立性，这也是数字逻辑电路分析中验证逻辑功能是否一致的基础方法：

真值表验证法：列出等式两边逻辑表达式的所有输入变量取值组合，对比对应的输出结果，若所有组合下输出结果均一致，则恒等式成立。该方法直观、严谨，是验证逻辑恒等式最基础的方法，尤其适用于变量数量较少（3 个及以内）的场景。

公式推导验证法：利用已证明的基本公式、常用公式与三大定理，对等式的一边进行变换、化简，若最终推导得到等式另一边的表达式，则恒等式成立。该方法适用于多变量、复杂逻辑表达式的验证，也是逻辑函数化简的核心方法。

2.公式与定理综合应用化简示例

通过一个完整的逻辑函数化简案例，展示基本公式、常用公式与三大定理的综合应用，在保证逻辑功能不变的前提下，将复杂表达式简化为最简与或形式：

待化简逻辑函数：$Y=ABC+A\overline{B}C+\overline{A}BC+\overline{A}C+AB+A$

1) 利用合并律，合并前两项：$ABC+A\overline{B}C=AC$，表达式简化为$Y=AC+\overline{A}BC+\overline{A}C+AB+A$
2) 利用吸收律 1，消去冗余项 AB：$A+AB=A$，表达式简化为$Y=AC+\overline{A}BC+\overline{A}C+A$
3) 利用吸收律 1，消去冗余项 AC：$A+AC=A$，表达式简化为$Y=\overline{A}BC+\overline{A}C+A$
4) 利用吸收律 2，消去冗余因子$\overline{A}$：$A+\overline{A}BC=A+BC$、$A+\overline{A}C=A+C$，表达式简化为$Y=A+BC+C$
5) 利用吸收律 1，消去冗余项 BC：$C+BC=C$，最终得到最简与或表达式：$Y=A+C$

示例：求解逻辑函数$Y=\left(A\overline{B}+C\right)\cdot\overline{D+E}$的反函数，分别用摩根定理多步推导与反演定理一步求解，对比反演定理的便捷性：

反演定理一步求解：按照规则变换，得到$\overline{Y}=\left(\overline{A}+B\right)\cdot\overline{C}+\overline{D}\cdot\overline{E}$

摩根定理多步推导验证：

$$\begin{aligned}\overline{Y} &= \overline{\left(A\overline{B}+C\right)\cdot\overline{D+E}} \\ &= \overline{A\overline{B}+C}+\overline{\overline{D+E}} \\ &= \overline{A\overline{B}}\cdot\overline{C}+D+E \\ &= \left(\overline{A}+B\right)\cdot\overline{C}+D+E\end{aligned}$$

二者结果完全一致，验证了反演定理的准确性与便捷性。

**五、优先级规则补充**

在复杂逻辑表达式的运算、化简与变换过程中，为保证运算结果的唯一性，需严格遵循逻辑代数的运算优先级规则，该规则与 1.1.1 节中基本逻辑运算的优先级完全一致，结合本节内容补充完整如下：

1.括号优先级最高：表达式中存在括号时，优先执行括号内的所有运算，多层括号遵循“从内到外”的运算顺序；

2.非运算优先级次之：非号覆盖的变量或表达式，优先执行取反运算，非号仅对其正下方覆盖的内容生效；

3.与运算优先级第三：与运算优先级高于或运算，无括号时，先执行所有与运算，再执行或运算；

4.异或、同或运算优先级第四：二者优先级介于与运算和或运算之间，工程设计中通常通过括号明确其运算顺序，避免歧义；

5.或运算优先级最低：无括号、无更高优先级运算时，按照从左到右的顺序执行或运算。

## 1.2 逻辑函数的公式化简法

在 1.1 节中，我们系统学习了逻辑代数的基本逻辑运算、基本公式、常用公式与三大基本定理，这些内容共同构成了逻辑代数的完整运算体系。而这套运算体系最核心的工程应用，就是实现逻辑函数的化简与变换。在数字逻辑电路的设计流程中，任何一个逻辑功能都可以用对应的逻辑函数来描述，而逻辑函数的复杂程度，直接决定了实现该功能的硬件电路的复杂度、制造成本、运行功耗与长期可靠性。公式化简法（也称为代数化简法），就是利用 1.1 节中学习的逻辑代数基本公式、常用公式与三大定理，对逻辑函数进行等价变换，消去表达式中的冗余项与冗余变量，最终得到最简逻辑表达式的方法，是数字逻辑电路设计中最基础、最核心的技能之一。

### 1.2.1 逻辑函数的表示方法

**一、逻辑函数化简的工程意义**

任何一个逻辑函数，都可以通过多种不同形式的表达式来描述，而每一种表达式都对应着一套具体的门电路实现方案。对于同一个逻辑功能，未经化简的逻辑函数往往包含大量的冗余项与冗余变量，直接基于该表达式设计电路，会出现门电路使用数量多、门电路输入端数量多、电路接线复杂、信号传输延迟长等问题。这些问题不仅会提升硬件电路的制造成本，还会增加电路的故障概率，降低系统的运行稳定性。

我们可以通过一个直观的案例理解化简的工程价值：对于逻辑函数$Y = AB + A\overline{B} + \overline{A}B$，未经化简时，该表达式包含 3 个与项，需要 3 个二输入与门、1 个三输入或门，共 4 个门电路实现；而通过公式化简后，该函数可简化为$Y = A + B$，仅需 1 个二输入或门即可实现完全一致的逻辑功能。化简后的电路门数量减少了 75%，接线复杂度大幅降低，信号传输延迟缩短，同时制造成本与故障概率也同步下降，这正是逻辑函数化简的核心工程价值。

**二、逻辑函数的常见表达形式**

逻辑函数的表达式并非只有单一形式，根据运算组合的不同，同一个逻辑函数可以呈现出多种不同的表达形式，其中最常用的有五种，分别为与-或表达式、或-与表达式、与非-与非表达式、或非-或非表达式、与-或-非表达式。五种表达式之间可通过逻辑代数的基本公式与定理实现等价转换，且均对应着不同的门电路实现方案。

我们以同一个逻辑函数为例，展示五种常见表达形式的对应关系：

1.与-或表达式

定义：由若干个与项通过或运算组合而成的表达式，是逻辑代数中最基础、最常用的表达式形式，也是公式化简的核心目标形式。

示例：$Y = AB + \overline{A}C$

电路对应：由多个与门的输出端接入一个或门实现，先与后或的运算顺序。

2.或-与表达式

定义：由若干个或项通过与运算组合而成的表达式，是与-或表达式的对偶形式。

示例：$Y = (A + C) \cdot (\overline{A} + B)$

电路对应：由多个或门的输出端接入一个与门实现，先或后与的运算顺序。

3.与非-与非表达式

定义：由若干个与非项通过与非运算组合而成的表达式，是工程中最常用的表达式形式之一，适配全与非门的电路设计。

示例：$Y = \overline{\overline{AB} \cdot \overline{\overline{A}C}}$

电路对应：全部由与非门实现，无需其他类型门电路，是通用逻辑芯片设计中最常用的形式。

4.或非-或非表达式

定义：由若干个或非项通过或非运算组合而成的表达式，是与非-与非表达式的互补形式，适配全或非门的电路设计。

示例：$Y=\overline{\overline{A+C}+\overline{\overline{A}+B}}$

电路对应：全部由或非门实现，在 CMOS 集成电路设计中应用广泛。

5.与-或-非表达式

定义：先对各组变量执行与运算，再执行或运算，最后整体取反的表达式，直接对应与或非门电路。

示例：$Y=\overline{A\overline{C}+\overline{A\overline{B}}}$

电路对应：直接由与或非门实现，无需额外的门电路级联，电路结构紧凑。

**三、逻辑函数的最简标准**

由于与-或表达式是逻辑函数最基础的表达形式，且其他所有形式的表达式都可以由最简与-或表达式等价转换而来，因此逻辑函数的最简标准，核心以最简与-或表达式为基准，其他形式的最简标准均可由此对应推导。

最简与-或表达式必须同时满足以下两个核心条件，二者缺一不可：

1.与项的个数最少：即表达式中通过或运算连接的与项数量最少。对应电路设计中，所需的与门数量最少，同时后级或门的输入端数量最少，是降低电路复杂度的核心指标。

2.每个与项中的变量个数最少：在满足与项个数最少的前提下，每个与项中包含的逻辑变量（原变量与反变量均计入）数量最少。对应电路设计中，每个与门的输入端数量最少，进一步简化电路的接线设计，降低信号传输的延迟。

需要特别强调的是，两个条件必须严格遵循“先保证与项个数最少，再保证每个与项的变量个数最少”的优先级顺序。若仅减少了变量个数，但增加了与项的数量，不属于有效的化简；只有同时满足两个条件的表达式，才是真正的最简与-或表达式。

### 1.2.2 公式化简法的核心基本方法

公式化简法的核心逻辑，是灵活运用 1.1 节中学习的基本公式与常用公式，对逻辑函数进行等价变换，消去冗余项与冗余变量。根据所用公式的不同，公式化简法可分为四种核心基本方法，分别为并项法、吸收法、消去法、配项法。在实际化简过程中，往往需要将四种方法组合使用，才能完成复杂逻辑函数的化简。

**一、并项法**

1.核心原理与公式依据

并项法的核心依据是 1.1 节中学习的合并律，公式为：

$$AB+A\overline{B}=A$$

结合代入定理，该公式可扩展为通用形式：两个与项中，除了一对互补的变量之外，其余所有变量完全相同，则这两个与项称为互补相邻项，可以合并为一个与项，合并后直接消去互补的变量，仅保留两个与项的公共变量。

并项法的核心优势，是可以将两个与项合并为一个，同时减少一个变量，直接降低表达式的复杂度，是公式化简中最基础、最常用的方法。

示例 1：化简逻辑函数$Y = ABC + AB\overline{C}$

化简过程：两个与项的公共变量为$AB$，互补变量为$C$和$\overline{C}$，根据合并律直接合并，得到：

$$Y = AB\left(C + \overline{C}\right) = AB \cdot 1 = AB$$

示例 2：化简逻辑函数$Y = A\overline{B}C + \overline{A}\overline{B}C$

化简过程：两个与项的公共变量为$\overline{B}C$，互补变量为$A$和$\overline{A}$，合并后得到：

$$Y = \overline{B}C\left(A + \overline{A}\right) = \overline{B}C \cdot 1 = \overline{B}C$$

示例 3：多变量复杂场景化简$Y = ABC\overline{D} + ABCD$

化简过程：两个与项的公共变量为$ABC$，互补变量为$D$和$\overline{D}$，合并后得到：

$$Y = ABC\left(\overline{D} + D\right) = ABC \cdot 1 = ABC$$

2.工程应用要点

并项法的核心是快速识别表达式中的互补相邻项，对于多变量逻辑函数，可先锁定公共变量，再寻找互补变量，无需展开表达式即可直接合并。在实际电路设计中，每完成一次并项操作，就可以减少一个与门的使用，同时降低后级或门的输入端数量，是最简标准中“减少与项个数”的核心实现方法。

**二、吸收法**

1.核心原理与公式依据

吸收法的核心依据是 1.1 节中学习的吸收律 1（消项公式），核心公式为：

$$A + AB = A$$

结合代入定理，该公式可扩展为通用形式：在与-或表达式中，如果一个与项是另一个与项的全部因子，那么包含该因子的长与项是完全冗余的，可以直接从表达式中消去，即短与项可以“吸收”掉长与项。吸收法的核心作用，是直接消去表达式中的冗余与项，快速减少与项的个数，是满足最简标准第一条件的核心方法。

示例 1：化简逻辑函数$Y = A + ABCD$

化简过程：与项$A$是与项$ABCD$的全部因子，根据吸收律，$ABCD$为冗余项，可直接消去，得到：

$$Y = A$$

示例 2：化简逻辑函数$Y = AB + ABC + ABD + AB\overline{CD}$

化简过程：与项$AB$是后续所有与项的全部公共因子，后续所有与项均为冗余项，可直接全部消去，得到：

$$Y = AB$$

示例 3：嵌套场景化简$Y = \overline{A} + \overline{A}BC + \overline{A}BD + \overline{AB}C$

化简过程：与项$\overline{A}$是后续所有与项的全部公共因子，后续所有与项均可被吸收，直接消去，得到：

$$Y = \overline{A}$$

2.工程应用要点

吸收法的核心是快速识别“短与项完全包含在长与项中”的关系，对于复杂表达式，可先锁定最短的与项，再检查其余与项是否包含该短与项的全部因子，无需复杂推导即可直接消去冗余项。在实际电路设计中，每消去一个冗余与项，就可以减少一个对应的与门电路，大幅简化电路结构。

**三、消去法**

1.核心原理与公式依据

消去法的核心依据是 1.1 节中学习的吸收律 2（消因子公式），核心公式为：

$$A + \overline{A}B = A + B$$

结合代入定理，该公式可扩展为通用形式：在与-或表达式中，如果一个与项的反变量，是另一个与项的其中一个因子，那么这个反变量因子是完全冗余的，可以直接从与项中消去。

消去法的核心作用，是在不改变与项个数的前提下，减少单个与项中的变量个数，是满足最简标准第二条件的核心方法。

示例 1：化简逻辑函数$Y = A + \overline{A}BC$

化简过程：与项$A$的反变量$\overline{A}$是与项$BC$的一个因子，$\overline{A}$为冗余因子，可直接消去，得到：

$$Y = A + BC$$

示例 2：化简逻辑函数$Y = AB + \overline{AB}CD$

化简过程：与项$AB$的反变量$\overline{AB}$是与项$CD$的一个因子，$\overline{AB}$为冗余因子，可直接消去，得到：

$$Y = AB + CD$$

示例 3：多变量嵌套场景化简$Y = \overline{A} + AB\overline{C}D$

化简过程：与项$\overline{A}$的反变量$A$是与项$B\overline{C}D$的一个因子，$A$为冗余因子，可直接消去，得到：

$$Y = \overline{A} + B\overline{C}D$$

2.工程应用要点

消去法的核心是识别“一个与项的整体，是另一个与项的一个反变量因子”的关系，对于复杂表达式，可先锁定单个变量的与项，再寻找包含其反变量的长与项，直接消去冗余因子。在实际电路设计中，每消去一个冗余变量，就可以减少对应与门的一个输入端，简化电路的接线设计，降低信号传输的接线复杂度。

**四、配项法**

1.核心原理与公式依据

配项法是公式化简法中的辅助方法，核心依据是互补律$A+\overline{A}=1$和重叠律$A+A=A$，核心逻辑为：当逻辑函数无法通过前三种方法直接化简时，可通过给某个与项乘以$A+\overline{A}$（等于 1，不改变表达式的逻辑功能），将一个与项拆分为两个与项，再结合前三种方法完成化简；也可通过重叠律，在表达式中重复加入某个与项，再分别与其他与项合并化简。

配项法的核心作用，是解决前三种方法无法直接处理的复杂逻辑函数，通过“先拆分、后化简”的方式，实现冗余项与冗余变量的消去。需要特别注意的是，配项法仅适用于前三种方法无法化简的场景，不可随意配项，否则会增加表达式的复杂度。

示例 1：基础配项化简逻辑函数$Y=AB+\overline{A}C+BC$

化简过程：该表达式无法通过并项、吸收、消去法直接化简，观察到第三项$BC$，可通过互补律配项，乘以$A+\overline{A}$（等于 1），表达式变为：

$$Y=AB+\overline{A}C+BC\left(A+\overline{A}\right)=AB+\overline{A}C+ABC+\overline{A}BC$$

再通过吸收法，$AB$吸收$ABC$，$\overline{A}C$吸收$\overline{A}BC$，最终得到最简表达式：

$$Y=AB+\overline{A}C$$

示例 2：重叠律配项化简逻辑函数$Y=A\overline{B}+\overline{A}B+AB+\overline{AB}$

化简过程：根据重叠律，$AB+AB=AB$，$\overline{AB}+\overline{AB}=\overline{AB}$，因此可在表达式中重复加入$AB$和$\overline{AB}$，表达式变为：

$$Y=\left(A\overline{B}+AB\right)+\left(\overline{A}B+\overline{A\overline{B}}\right)$$

再通过并项法分别合并，得到：

$$Y=A\left(\overline{B}+B\right)+\overline{A}\left(B+\overline{B}\right)=A+\overline{A}=1$$

示例 3：多变量复杂场景化简$Y=ABC+AB\overline{C}+A\overline{B}C+\overline{A}BC$

化简过程：通过重叠律，重复加入与项$ABC$，表达式变为：

$$Y=\left(ABC+AB\overline{C}\right)+\left(ABC+A\overline{B}C\right)+\left(ABC+\overline{A}BC\right)$$

再通过并项法分别合并，得到：

$$Y=AB+AC+BC$$

2.工程应用要点

配项法的核心是“以退为进”，通过合理的配项，创造出可使用前三种方法化简的条件。在实际应用中，配项需遵循“最小配项原则”，仅拆分必要的与项，避免过度配项导致表达式复杂度提升。对于初学者而言，配项法的难点在于选择合适的配项对象，可优先选择包含公共变量最多的与项进行配项，提升化简效率。

### 1.2.3 公式化简综合实例

在实际的数字电路设计中，绝大多数逻辑函数都是多变量、多与项的复杂表达式，仅依靠单一的化简方法无法完成化简，需要将并项法、吸收法、消去法、配项法四种核心方法组合使用，按照固定的步骤逐步化简，才能得到最简与-或表达式。

**一、综合公式化简的通用标准步骤**

对于任意复杂的与-或逻辑函数，可按照以下优先级顺序执行化简，确保化简过程有序、高效，避免遗漏冗余项与冗余变量：

1.执行吸收法，消去冗余与项

优先检查表达式中是否存在可被吸收的长与项，即是否存在一个与项是另一个与项的全部因子，直接消去所有可被吸收的冗余与项，最大限度减少与项的个数，优先满足最简标准的第一条件。

2.执行消去法，消去冗余变量因子

完成冗余与项消去后，检查剩余的与项中是否存在可消去的冗余反变量因子，直接消去所有冗余因子，减少每个与项中的变量个数，满足最简标准的第二条件。

3.执行并项法，合并互补相邻项

完成前两步化简后，检查剩余的与项中是否存在互补相邻项，将可合并的与项全部合并，进一步减少与项的个数与变量个数。

4.执行配项法，处理无法化简的剩余项

若经过前三步后，表达式仍无法达到最简状态，且存在无法合并、消去的与项，再通过配项法创造化简条件，完成最终的化简。

5.校验最简性，重复化简步骤

完成一轮化简后，再次按照前三步的顺序检查表达式，确认是否存在新的可吸收、可消去、可合并的项，重复化简步骤，直到无法再进行任何化简为止，确保得到最简与-或表达式。

**二、综合化简示例**

以下示例均为数字电路设计中常见的多变量逻辑函数，严格按照上述通用步骤执行化简，完整展示综合化简的全流程。

示例 1：化简四变量逻辑函数$Y = ABC + A\overline{B}C + \overline{A}C + BD + \overline{B}C\overline{D} + ABCD$

化简全流程：

1.执行吸收法，消去冗余与项

检查发现，与项$ABC$是与项$ABCD$的全部因子，因此$ABCD$为冗余项，直接消去，表达式简化为：

$$Y = ABC + A\overline{B}C + \overline{A}C + BD + \overline{B}C\overline{D}$$

2.执行消去法，消去冗余变量因子

检查剩余与项，未发现可直接消去的冗余反变量因子，进入第三步。

3.执行并项法，合并互补相邻项

检查发现，$ABC$与$A\overline{B}C$为互补相邻项，公共变量为$AC$，互补变量为$B$和$\overline{B}$，合并后得到$AC$，表达式简化为：

$$Y = AC + \overline{A}C + BD + \overline{B}C\overline{D}$$

再次检查，$AC$与$\overline{A}C$为互补相邻项，合并后得到$C$，表达式进一步简化为：

$$Y = C + BD + \overline{B}C\overline{D}$$

4.再次执行吸收法，消去冗余与项

检查发现，与项$C$是与项$\overline{B}C\overline{D}$的全部因子，因此$\overline{B}C\overline{D}$为冗余项，直接消去，表达式简化为：

$$Y = C + BD$$

5.最简性校验

再次检查，表达式仅包含 2 个与项，每个与项的变量个数最少，无法再进行任何化简，最终得到最简与-或表达式：

$$Y = C + BD$$

示例 2：化简五变量逻辑函数$Y = A\overline{B} + \overline{A}B + B\overline{C} + \overline{B}C + ACD$

化简全流程：

1.执行吸收法，消去冗余与项

初步检查，未发现可直接吸收的冗余与项，进入第二步。

2.执行消去法，消去冗余变量因子

检查发现，前两项$A\overline{B} + \overline{A}B$与第五项$ACD$，可通过消去法处理：$A\overline{B} + ACD = A\overline{B} + CD$，表达式简化为：

$$Y = A\overline{B} + \overline{A}B + B\overline{C} + \overline{B}C + CD$$

3.执行并项法，合并互补相邻项

检查发现，无直接可合并的互补相邻项，进入第四步配项法。

4.执行配项法，创造化简条件

观察表达式，前四项为循环相邻结构，对$B\overline{C}$项配项，乘以$A+\overline{A}$，表达式变为：

$$Y=A\overline{B}+\overline{A}B+B\overline{C}\left(A+\overline{A}\right)+\overline{B}C+CD$$

展开后得到：

$$Y=A\overline{B}+\overline{A}B+AB\overline{C}+\overline{A}B\overline{C}+\overline{B}C+CD$$

5.重复执行吸收法、并项法

吸收法：$A\overline{B}$吸收$AB\overline{C}$，得到$A\overline{B}+B\overline{C}$；$\overline{A}B$吸收$\overline{A}B\overline{C}$，无变化；

并项法：$A\overline{B}+\overline{B}C$无法合并，$\overline{A}B+B\overline{C}$无法合并，再次通过冗余律检查，发现$\overline{A}B+\overline{B}C=\overline{A}B+\overline{B}C+\overline{A}C$，加入$\overline{A}C$项后，可与$CD$通过消去法处理：$\overline{A}C+CD=\overline{A}C+D$；

最终经过多轮化简，得到最简与-或表达式：

$$Y=A\overline{B}+B\overline{C}+\overline{A}C+D$$

### 1.2.4 逻辑函数的表达式形式转换

在实际的工程设计中，我们通常先将逻辑函数化简为最简与-或表达式，再根据实际使用的门电路类型，将其转换为对应的表达式形式。最常见的转换需求，是将最简与-或表达式转换为与非-与非表达式、与-或-非表达式、或-与表达式、或非-或非表达式，四种转换均基于 1.1 节中学习的摩根定理、反演定理与对偶定理，转换过程均为等价变换，不会改变原逻辑函数的功能。

**一、最简与-或表达式转换为最简与非-与非表达式**

这是工程设计中最常用的转换形式，适配全与非门的电路设计，是通用数字逻辑芯片最常用的表达式形式。

1.转换原理

转换的核心依据是还原律与摩根定理，核心逻辑为：对最简与-或表达式连续执行两次取反运算，根据还原律，表达式的逻辑功能不变；再通过摩根定理，将内层的取反运算转换为与非形式，最终得到与非-与非表达式。

2.转换步骤

1.先将逻辑函数化简为最简与-或表达式；

2.对最简与-或表达式整体执行两次取反运算，外层非号保持不变，内层非号覆盖整个与-或表达式；

3.利用摩根定理，将内层的“或运算的反”转换为“各与项反的与运算”，最终得到最简与非-与非表达式。

示例：

将逻辑函数$Y=AB+\overline{A}C$转换为最简与非-与非表达式。

第一步：该表达式已为最简与-或表达式；

第二步：对表达式整体两次取反，得到$Y=\overline{\overline{AB+\overline{A}C}}$；

第三步：利用摩根定理对内层取反运算转换，$\overline{AB+\overline{A}C}=\overline{AB}\cdot\overline{\overline{A}C}$，代入后得到最终的最简与非-与非表达式：

$$Y=\overline{\overline{AB}\cdot\overline{\overline{A}C}}$$

电路对应：该表达式仅需 3 个二输入与非门即可实现，无需其他类型门电路，完全适配全与非门的设计需求。

**二、最简与-或表达式转换为最简与-或-非表达式**

该转换形式直接对应与或非门电路，电路结构紧凑，信号传输延迟短，在集成电路设计中应用广泛。

1.转换原理

转换的核心依据是反演定理与互补律，核心逻辑为：先通过反演定理求出原函数的反函数$\overline{Y}$，并将反函数化简为最简与-或表达式；再对反函数整体取反，得到的$\overline{\overline{Y}}$就是原函数的最简与-或-非表达式。

2.转换步骤

1.先将原函数化简为最简与-或表达式；

2.利用反演定理，求出原函数的反函数$\overline{Y}$，并将反函数化简为最简与-或表达式；

3.对化简后的反函数整体取反，得到原函数的最简与-或-非表达式。

示例：将逻辑函数$Y=AB+\overline{A}C$转换为最简与-或-非表达式。

第一步：该表达式已为最简与-或表达式；

第二步：利用反演定理求反函数，$\overline{Y}=\left(\overline{A}+\overline{B}\right)\cdot\left(A+\overline{C}\right)=\overline{A\overline{C}}+A\overline{B}$，该式为反函数的最简与-或表达式；

第三步：对反函数整体取反，得到最终的最简与-或-非表达式：

$$Y=\overline{\overline{A\overline{C}}+A\overline{B}}$$

电路对应：该表达式仅需 1 个二组二输入与或非门即可实现，电路结构极为紧凑。

**三、最简与-或表达式转换为最简或-与表达式**

该转换形式适配或门-与门的电路结构，是转换为或非-或非表达式的中间基础。

1.转换核心原理

转换的核心依据是反演定理，核心逻辑为：先求出原函数的反函数的最简与-或表达式，再对反函数的最简与-或表达式应用反演定理，得到的结果就是原函数的最简或-与表达式。

2.转换标准步骤

1.先将原函数化简为最简与-或表达式；

2.利用反演定理，求出原函数的反函数$\overline{Y}$，并化简为最简与-或表达式；

3.对反函数的最简与-或表达式再次应用反演定理，得到原函数的最简或-与表达式。

示例：

将逻辑函数$Y = AB + \overline{A}C$转换为最简或-与表达式。

第一步：该表达式已为最简与-或表达式；

第二步：求反函数的最简与-或表达式，$\overline{Y} = \overline{A\overline{C}} + A\overline{B}$；

第三步：对反函数应用反演定理，得到原函数的最简或-与表达式：

$$Y = (A + C) \cdot \left(\overline{A} + B\right)$$

电路对应：该表达式需 2 个二输入或门、1 个二输入与门实现。

**四、最简与-或表达式转换为最简或非-或非表达式**

该转换形式适配全或非门的电路设计，在 CMOS 集成电路设计中应用广泛。

1.转换原理

转换的核心依据是还原律与摩根定理，核心逻辑为：先将原函数转换为最简或-与表达式，再对最简或-与表达式连续执行两次取反运算，通过摩根定理将内层的取反运算转换为或非形式，最终得到或非-或非表达式。

2.转换步骤

1.先将原函数化简为最简与-或表达式，再转换为最简或-与表达式；

2.对最简或-与表达式整体执行两次取反运算，外层非号保持不变，内层非号覆盖整个或-与表达式；

3.利用摩根定理，将内层的“与运算的反”转换为“各或项反的或运算”，最终得到最简或非-或非表达式。

示例：

将逻辑函数$Y = AB + \overline{A}C$转换为最简或非-或非表达式。

第一步：先转换为最简或-与表达式，$Y = (A + C) \cdot \left(\overline{A} + B\right)$；

第二步：对表达式整体两次取反，得到$Y = \overline{\overline{(A + C) \cdot \left(\overline{A} + B\right)}}$；

第三步：利用摩根定理对内层取反运算转换，$\overline{(A + C) \cdot \left(\overline{A} + B\right)} = \overline{A + C} + \overline{\overline{A} + B}$，代入后得到最终的最简或非-或非表达式：

$$Y = \overline{\overline{A + C} + \overline{\overline{A} + B}}$$

电路对应：该表达式仅需 3 个二输入或非门即可实现，完全适配全或非门的设计需求。

## 1.3 逻辑函数的图形化简法

在 1.2 节中，我们系统学习了逻辑函数的公式化简法，该方法通用性强，不受变量数量限制，但也存在明显的局限性：化简过程高度依赖设计者对逻辑代数公式的熟练程度与工程经验，对于复杂多变量逻辑函数，不仅化简过程繁琐，还难以直观判断最终结果是否达到真正的最简状态，甚至可能出现多种化简结果无法取舍的情况。

为了解决公式化简法的上述问题，工程领域普遍采用图形化简法，也称为卡诺图（Karnaugh Map）化简法。该方法由美国电信工程师莫里斯·卡诺于 1953 年提出，核心是将逻辑函数的最小项以图形化的方式排列，利用几何位置的相邻性直观体现逻辑最小项的相邻性，通过图形合并的方式直接消去冗余变量，无需复杂的公式推导，就能快速、准确地得到逻辑函数的最简表达式，是数字逻辑电路分析与设计中最核心、最常用的化简方法。

### 1.3.1 最小项与卡诺图

**一、最小项的定义**

在 n 变量逻辑函数中，若一个乘积项（与项）包含了全部 n 个逻辑变量，每个变量都以原变量或反变量的形式在该乘积项中出现且仅出现一次，则这个乘积项被称为该 n 变量逻辑函数的最小项。

例如二变量逻辑函数的输入变量为 A、B，其全部最小项为$\overline{A}\overline{B}$、$\overline{A}B$、$A\overline{B}$、$AB$，共$2^2=4$个；三变量逻辑函数的输入变量为 A、B、C，共有$2^3=8$个最小项；以此类推，n 变量逻辑函数共有$2^n$个互不重复的最小项。

**二、最小项的编号**

为了便于书写与图形化表示，每个最小项都有唯一的编号，用$m_i$表示，其中 m 代表最小项，下标 i 为该最小项对应的十进制编号。

编号规则：将最小项中的原变量记为 1，反变量记为 0，按约定的变量高位到低位顺序排列得到一组二进制数，该二进制数对应的十进制数，就是这个最小项的编号 i。例如三变量逻辑函数约定 A 为最高位、C 为最低位，最小项$\overline{A}BC$对应的二进制数为 011，十进制数为 3，因此编号为$m_3$；最小项$AB\overline{C}$对应的二进制数为 110，十进制数为 6，编号为$m_6$。

**三、最小项的核心性质**

最小项是卡诺图化简的核心基础，具备以下 4 个关键性质，也是化简的核心原理：

1.唯一性：对于任意一个最小项，只有唯一的一组输入变量取值组合能使其值为 1；对于其他所有取值组合，该最小项的值均为 0。换句话说，输入变量的每一组取值，都对应且仅对应一个最小项的值为 1。

2.互斥性：对于同一组输入变量取值，任意两个不同的最小项的乘积恒为 0。因为同一组取值下，不可能有两个最小项同时为 1，因此相与结果必然为 0。

3.完备性：n 变量逻辑函数的全部$2^n$个最小项的和（或运算）恒为 1。因为对于任意一组输入取值，总有且仅有一个最小项为 1，因此全部最小项相或的结果恒为 1。

4.相邻性：若两个最小项之间，仅有一个变量互为反变量，其余所有变量均完全相同，则称这两个最小项为逻辑相邻项，简称相邻项。相邻的两个最小项可以合并为一个乘积项，并消去那个互补的变量。例如三变量最小项$m_6(AB\overline{C})$和$m_7(ABC)$，仅变量 C 互为反变量，因此可以合并为$AB\overline{C}+ABC=AB$，直接消去了互补变量 C。这一性质，正是卡诺图化简法的核心原理。

**四、逻辑函数的最小项表达式**

全部由最小项相或构成的逻辑函数表达式，称为最小项表达式，也叫标准与-或表达式，是逻辑函数的标准形式之一，也是填入卡诺图的基础。任何一个逻辑函数，都可以通过基本公式与定理，转换为唯一的最小项表达式，转换方法如下：

1.先将逻辑函数转换为一般与-或表达式；

2.对于缺少变量的非标准与项，利用互补律$A+\overline{A}=1$，给该与项乘以缺少的变量的，将其拆分为若干个标准最小项；

3.利用重叠律$A+A=A$，合并重复的最小项，最终得到标准最小项表达式。

例如将逻辑函数$Y=AB+\overline{A}C$转换为最小项表达式：

第一步，该式已为与-或表达式，第一个与项 AB 缺少变量 C，第二个与项$\overline{A}C$缺少变量 B；

第二步，给 AB 乘以$(C+\overline{C})$，给$\overline{A}C$乘以$(B+\overline{B})$，展开得到：

$$Y=m_1+m_3+m_6+m_7=\sum m(1,3,6,7)$$

其中$\sum$表示对括号内编号的最小项进行或运算。

### 1.3.2 卡诺图的结构与绘制

卡诺图是将 n 变量逻辑函数的全部$2^n$个最小项，按照逻辑相邻性等价于几何相邻性的原则，排列成的矩形方格阵列。每个方格对应一个唯一的最小项，方格的几何位置相邻，就代表对应的最小项逻辑相邻，实现了逻辑关系的可视化。

卡诺图的核心设计原则：行、列变量的取值必须采用格雷码（循环码）的方式排列，而非自然二进制码。格雷码的特点是相邻两组编码之间仅有一位二进制数不同，其余位完全相同，这样就能保证：卡诺图中几何位置上相邻（上下、左右）的方格，对应的最小项必然是逻辑相邻项；同时，卡诺图具备循环相邻特性，即最上一行与最下一行、最左一列与最右一列的方格也为逻辑相邻，这是初学者容易忽略的关键特性。

二变量逻辑函数共有 4 个最小项，对应 2 行 2 列的方格阵列，通常将变量 A 作为行变量变量 B 作为列变量，二变量格雷码与自然二进制码一致，结构如下图：

图 1-3-1　二变量卡诺图结构

行变量 A 的取值为 0、1，列变量 B 的取值为 0、1，每个方格内标注对应最小项的编号，相邻方格的最小项均为逻辑相邻项。

三变量逻辑函数共有 8 个最小项，对应 2 行 4 列的方格阵列，通常将变量 A 作为行变量，变量 B、C 作为列变量，列变量 BC 的取值必须按格雷码 00、01、11、10 排列，保证相邻列的最小项逻辑相邻，结构如下：

图 1-3-2　三变量卡诺图结构

四变量逻辑函数共有 16 个最小项，是工程设计中最常用的类型，对应 4 行 4 列的方格阵列，通常将变量 A、B 作为行变量，变量 C、D 作为列变量，行、列取值均按格雷码排列，结构如下：

图 1-3-3　四变量卡诺图结构

### 1.3.3　逻辑函数的卡诺图表示

要使用卡诺图化简逻辑函数，首先需要将逻辑函数填入对应的卡诺图中，核心原则是：在逻辑函数中出现的最小项，对应卡诺图的方格内填入 1；未出现的最小项，对应方格内填入 0。根据逻辑函数的不同表达形式，有三种常用的填入方法。

**一、由最小项表达式填入卡诺图**

这是最直接的填入方法，步骤如下：

1.绘制对应变量数的卡诺图；

2.在卡诺图中，对应最小项编号的方格内填入 1，其余方格内填入 0。

例如逻辑函数$Y=\sum m(1,3,6,7)$为三变量函数，绘制三变量卡诺图，在$m_1$、$m_3$、$m_6$、$m_7$对应的方格内填入 1，其余填入 0，得到的卡诺图如下：

图 1-11 逻辑函数$Y=\sum m(1,3,6,7)$的卡诺图：

图 1-3-4　卡诺图结构

**二、由一般与-或表达式直接填入卡诺图**

对于未转换为最小项表达式的一般与-或表达式，无需展开为最小项，可直接根据与运算“全 1 出 1”的规则填入卡诺图，大幅简化填入过程，步骤如下：

1.绘制对应变量数的卡诺图，先将所有方格初始化为 0；

2.对表达式中的每一个与项，找到卡诺图中所有满足该与项取值为 1 的方格，在这些方格内填入 1；

3.所有与项处理完毕后，得到的就是该逻辑函数的卡诺图，重复填入 1 的方格仍为 1，符合重叠律。

例如逻辑函数$Y=AB+\overline{A}C$，直接填入三变量卡诺图：与项 AB 对应 A=1、B=1 的两个方格，填入 1；与项$\overline{A}C$对应 A=0、C=1 的两个方格，填入 1，最终得到的卡诺图与图 1-3-4 完全一致。

**三、由真值表填入卡诺图**

若已知逻辑函数的真值表，可直接对应填入卡诺图：真值表中每一组输入变量取值对应的输出结果为 1 时，在卡诺图对应最小项的方格内填入 1；输出结果为 0 时，填入 0。该方法无需任何转换，是初学者最容易掌握的填入方式，完全匹配数字电路“真值表-卡诺图-电路设计”的标准流程。

### 1.3.4　卡诺图化简规则与步骤

一、原理：

卡诺图化简的核心原理，源于最小项的相邻性：两个逻辑相邻的最小项相或，可以合并为一个乘积项，并消去那个互补的变量；以此类推，$2^n$个相邻的最小项构成的矩形阵列，可以合并为一个乘积项，消去 n 个互补变量。

合并的核心规律：合并的最小项数量必须是 2 的整数次幂（2、4、8、16……），分别对应消去 1、2、3、4 个变量；合并的最小项必须构成连续的矩形阵列，且满足卡诺图的循环相邻规则。例如 2 个相邻最小项$m_6(AB\overline{C})+m_7(ABC)$可合并为 AB，消去变量 C；4 个相邻最小项$m_0$、$m_1$、$m_2$、$m_3$可合并为$\overline{AB}$，消去变量 C、D。

二、卡诺图化简的合并规则（画圈规则）：

卡诺图化简的核心操作，是对填入 1 的方格进行画圈合并，每个圈对应一个合并后的乘积项，最终所有圈对应的乘积项相或，就是逻辑函数的最简与-或表达式。画圈必须严格遵循以下 6 条规则：

1.圈定范围规则：每个圈内只能包含填入 1 的方格，不能包含填入 0 的方格；圈内的 1 方格数量必须是 2 的整数次幂，且必须构成矩形阵列，不能是不规则图形。

2.相邻性规则：圈内的所有 1 方格必须满足逻辑相邻性，包括上下、左右相邻，以及卡诺图的循环相邻。

3.最大化规则：每个圈的范围应尽可能大，圈内包含的 1 方格数量应尽可能多。圈越大，合并后消去的变量越多，对应的乘积项越简单。

4.全覆盖规则：卡诺图中所有填入 1 的方格，必须被至少一个圈覆盖，不能有遗漏的 1 方格，否则会丢失逻辑功能。

5.最少圈数规则：在满足全覆盖和最大化规则的前提下，圈的总数量应尽可能少。圈的数量越少，最终最简表达式中的与项个数越少。

6.重叠性规则：同一个 1 方格可以被多个圈重复覆盖，但每个圈必须至少包含一个未被其他圈覆盖的、独有的 1 方格，否则这个圈就是冗余圈，必须删除。

三、卡诺图化简的标准步骤：

1.绘制对应变量数量的标准卡诺图；

2.根据逻辑函数的表达形式，将其填入卡诺图中，出现的最小项填 1，其余填 0；

3.严格遵循画圈规则，对所有填 1 的方格进行画圈合并，优先画最大的圈，再画小圈，保证圈数最少、全覆盖、无冗余圈；

4.写出每个圈对应的乘积项：保留圈内取值不变的变量（原变量记为 1，反变量记为 0），消去取值发生变化的互补变量；

5.将所有圈对应的乘积项相或，得到逻辑函数的最简与-或表达式；

6.校验最简性，若存在冗余项，返回第三步调整画圈方式。

示例：用卡诺图化简四变量逻辑函数$Y=\sum_m(1，3，4，5，10，11，12，13)$

画圈合并：由卡诺图 1-12 可知，把可以合并的最小项圈出来；

写出最简与-或表达式：$Y=B\overline{C}+\overline{A}\overline{B}D+A\overline{B}C$。

图 1-3-5　卡诺图画圈示例

## 1.4　具有约束的逻辑函数的化简

在前两节的学习中，我们所接触的逻辑函数均为完全定义的逻辑函数——对于 n 个输入变量的全部$2^n$种取值组合，每一种组合都对应着确定的输出逻辑值（0 或 1），输入变量的取值不受任何限制，可以自由组合。但在实际的数字系统工程设计中，绝大多数逻辑函数都会受到实际应用场景的限制：输入变量的取值并非完全任意，某些取值组合在系统正常工作中绝对不会出现，或是某些取值组合出现时，输出结果为 0 或 1 都不会影响系统的核心逻辑功能。这类存在输入取值限制的逻辑函数，被称为不完全定义的逻辑函数，也叫具有约束的逻辑函数。

针对这类逻辑函数，若沿用常规的公式化简法与卡诺图化简法，不考虑输入约束的特性，往往无法得到最简的逻辑表达式，甚至会设计出冗余、复杂的硬件电路。本节将系统讲解逻辑约束的本质、约束项与无关项的定义，以及具有约束的逻辑函数的两种核心化简方法，重点讲解工程中最常用的、结合约束特性的卡诺图化简法，实现不完全定义逻辑函数的最简设计。

### 1.4.1　逻辑函数中的约束与约束条件

逻辑函数中的约束，本质是输入逻辑变量之间的相互制约关系，这种制约关系完全来源于实际工程场景的物理限制与功能要求，而非逻辑代数本身的数学规则。约束条件是描述输入变量之间制约关系的逻辑表达式，是具有约束的逻辑函数化简的核心基础。

从逻辑代数的角度看，约束所对应的、不允许出现的输入取值组合，其对应的最小项在系统正常工作中恒等于 0——因为这些输入组合永远不会出现，所以对应的最小项永远不会取到 1。基于这一特性，约束条件的通用逻辑表达形式为：所有约束对应的最小项相或，结果恒等于 0。

我们以 8421BCD 码为例，其无效编码对应的最小项为$m_{10}$~$m_{15}$，因此该系统的约束条件可表示为：

$$\sum d(10,11,12,13,14,15) = 0$$

约束条件的核心特性是：由于约束项的和恒为 0，因此在逻辑函数表达式中，任意加入一个或多个约束项，都不会改变原逻辑函数的输出结果——这正是具有约束的逻辑函数能够进一步化简的核心原理，也是区别于常规逻辑函数化简的关键。

### 1.4.2 约束项、任意项和无关项

**一、约束项与任意项**

在不完全定义的逻辑函数中，有两类特殊的最小项，分别为约束项与任意项，二者共同构成了逻辑化简中的无关项。

约束项是由输入变量之间的互斥关系、编码体系限制所决定的、在系统正常工作中绝对不会出现的输入取值组合所对应的最小项。其核心特征是：无论系统输出如何定义，这些最小项的取值恒为 0，永远不会对有效逻辑功能产生影响。例如前文提到的电机控制电路中，“正转、反转同时有效”对应的输入组合$AB = 11$，其对应的最小项$ABC$、$AB\overline{C}$均为约束项；8421BCD 码中的 6 个无效编码对应的最小项，也均为约束项。

任意项是另一类特殊的最小项：这类输入取值组合在系统中是可以出现的，但该组合出现时，输出逻辑值为 0 还是 1，对整个系统的核心功能、运行状态没有任何影响，输出可以任意定义。例如在 8421BCD 码的十进制数显示驱动电路中，当输入为无效编码时，数码管无论显示什么内容，都不会影响系统的正常计数功能，这些无效编码对应的最小项就属于任意项。在部分教材中，任意项也被称作“随意项”。

**二、无关项的统一定义**

约束项和任意项虽然来源不同，但在逻辑函数化简中具备完全相同的特性：我们可以根据化简的需要，自由地将其当作逻辑 1 或逻辑 0 来处理，既不会改变原逻辑函数的有效功能，又能最大限度地简化逻辑表达式。因此，工程上将约束项和任意项统称为逻辑无关项，简称无关项，用符号$d_i$表示，其中 i 为对应最小项的十进制编号。

**三、无关项的三种标准表示方法**

在数字电路设计中，无关项有三种通用的标准表示方法，分别适配逻辑表达式书写、卡诺图化简、工程文档标注三种场景：

1.最小项表达式标注法

这是逻辑函数的标准书写形式，用$\sum m(\ldots)$表示有效输出为 1 的最小项，用$\sum d(\ldots)$表示无关项，二者之间用“+”连接。例如 8421BCD 码的奇数校验电路，有效输出为 1 的最小项为$m_1$、$m_3$、$m_5$、$m_7$、$m_9$，无关项为$m_{10}$~$m_{15}$，其逻辑函数表达式为：

$$Y = \sum m(1,3,5,7,9) + \sum d(10,11,12,13,14,15)$$

$$\sum d(10,11,12,13,14,15) = 0$$

2.约束条件表达式法

该方法用于明确标注系统的约束限制，通常与逻辑函数表达式配合使用，核心形式为所有无关项的或运算恒等于 0。

3.卡诺图标注法

该方法适配卡诺图化简法，是工程中最直观的表示方式。在卡诺图中，有效最小项对应的方格填入 1，无效最小项填入 0，无关项对应的方格填入符号“×”（或“Φ”），用于区分有效项与无关项，便于化简时灵活处理。

### 1.4.3 含约束条件的化简方法

具有约束的逻辑函数的化简，核心是利用无关项“可自由当作 1 或 0 处理”的特性，在不改变有效逻辑功能的前提下，最大限度地简化逻辑表达式。常用的化简方法分为两种：结合约束条件的公式化简法与结合无关项的卡诺图化简法，其中卡诺图化简法直观、简便，不易出错，是工程设计中的首选方法。

**一、结合约束条件的公式化简法**

公式化简法的核心原理，是利用约束条件$\sum d\,i$=0 的特性：由于多个约束项的和恒为 0，因此可以在原逻辑函数的与-或表达式中，任意加入一个或多个约束项，不会改变原函数的输出结果。加入约束项后，可利用 1.2 节学习的合并律、吸收律、消去法等公式，合并同类项、消去冗余变量，得到比常规化简更简洁的最简与-或表达式。

化简步骤

1.写出原逻辑函数的标准与-或表达式，以及对应的约束条件；

2.根据化简需要，在原表达式中加入合适的约束项（加入的约束项需能与原有的有效项形成互补相邻项，便于合并化简）；

3.利用逻辑代数的基本公式与常用公式，对加入约束项后的表达式进行化简；

4.校验化简结果，确保有效逻辑功能不变，得到最简与-或表达式。

以 8421BCD 码的奇数校验电路为例，原逻辑函数为 Y=$\sum m$(1,3,5,7,9)，约束条件为$\sum d$(10,11,12,13,14,15)=0，用公式法化简该函数。

1.先写出原函数的与-或展开式：

$$Y=\overline{A\overline{B}\overline{C}}D+\overline{A\overline{B}}CD+\overline{A}B\overline{C}D+\overline{A}BCD+A\overline{B}\overline{C}D$$

2.观察约束项，选择能与原有效项合并的约束项$m_{11}(A\overline{B}CD)$、$m_{13}(AB\overline{C}D)$、$m_{15}(ABCD)$加入原表达式（约束项和为 0，加入不改变原函数）：

$$Y=\overline{A\overline{B}\overline{C}}D+\overline{A\overline{B}}CD+\overline{A}B\overline{C}D+\overline{A}BCD+A\overline{B}\overline{C}D+A\overline{B}CD+AB\overline{C}D+ABCD$$

3.利用合并律分组合并：

第一组：$\overline{ABC}D+\overline{AB}CD=\overline{AB}D$

第二组：$\overline{A}B\overline{C}D+\overline{A}BCD=\overline{A}BD$

第三组：$A\overline{BC}D+A\overline{B}CD=A\overline{B}D$

第四组：$AB\overline{C}D+ABCD=ABD$

4.再次合并化简：

$$Y=\overline{A\overline{B}}D+\overline{A}BD+A\overline{B}D+ABD=\overline{A}D\left(\overline{B}+B\right)+AD\left(\overline{B}+B\right)=\overline{A}D+AD=D$$

5.结果校验：原函数的有效逻辑为“输入 8421BCD 码为奇数时输出 1”，而 8421BCD 码的奇数的最低位 D 恒为 1，偶数的最低位 D 恒为 0，因此化简结果$Y=D$完全符合原逻辑功能，且为最简表达式。

若不利用约束条件，常规公式化简的结果为$Y=\overline{A}D+A\overline{BC}D$，远复杂于利用约束条件得到的最简结果，充分体现了约束项在公式化简中的核心价值。

**二、结合无关项的卡诺图化简法**

卡诺图化简法是处理具有约束的逻辑函数的首选方法，它无需复杂的公式推导，可通过图形化的方式直观判断哪些无关项可以当作 1 处理，快速实现圈的最大化、圈数最小化，得到最简表达式，完全规避了公式化简中“如何选择合适的约束项”的难点。

结合无关项的卡诺图化简，核心规则与常规卡诺图化简基本一致，仅补充 4 条针对无关项的专属规则：

1.卡诺图中，有效最小项填 1，无效最小项填 0，无关项填“×”；

2.画圈合并时，可将“×”当作 1 处理，也可当作 0 处理，唯一的判断标准是：能否让圈的范围更大、圈的总数量更少，以此实现最简化简；

3.每个合并圈必须至少包含一个有效的 1 方格，禁止出现全由“×”组成的圈，否则会引入与有效逻辑无关的冗余项，破坏原函数的逻辑功能；

4.未被圈入任何合并圈的“×”，一律当作 0 处理，避免引入不必要的逻辑项。

标准化简步骤：

1.绘制对应变量数量的标准卡诺图，在有效最小项方格内填 1，无关项方格内填“×”，其余方格填 0；

2.优先寻找能同时覆盖最多 1 方格和×方格的合并圈，严格遵循“圈的大小为 2 的整数次幂、矩形阵列、循环相邻”的规则，实现圈的最大化；

3.用最少的圈覆盖所有的有效 1 方格，每个圈至少包含一个独有的 1 方格，避免冗余圈；

4.写出每个合并圈对应的乘积项，保留圈内取值不变的变量，消去取值变化的变量；

5.将所有乘积项相或，得到逻辑函数的最简与-或表达式，校验有效逻辑功能的一致性。

## 1.5 逻辑代数的基础知识测验题

一、单项选择题

1. 逻辑代数中，逻辑变量的取值有（）种。

A. 1　　B. 2　　C. 10　　D. 16

2. 以下表达式中，符合逻辑代数运算规律的是（）。

A. 1+1=2　　B. A+1=A　　C. A・0=0　　D. A+A=0

3. 与非运算的核心逻辑规律是（）。

A.全 1 出 1，有 0 出 0　　B.全 1 出 0，有 0 出 1

C.全 0 出 1，有 1 出 0　　D.全 0 出 0，有 1 出 1

4. 3 个输入变量的逻辑函数，共有（）个最小项。

A. 3　　B. 4　　C. 8　　D. 16

5. 最简与-或表达式的判定标准是（）。

A.乘积项个数最少　　B.每个乘积项变量数最少

C.乘积项个数最少，且每个乘积项变量数最少　　D.表达式长度最短

6. 卡诺图中，行、列变量的取值采用循环码排列，核心目的是（）。

A.便于绘制　　B.保证几何相邻对应逻辑相邻　　C.减少方格数量　　D.符合二进制规则

7. 含约束项的逻辑函数化简中，约束项（）。

A.必须视为 0　　B.必须视为 1　　C.可任意视为 0 或 1　　D.不能参与化简

8. 以下表达式中，不属于最小项的是（）。

A. $AB$　　B. $ABC$　　C. $AB+C$

9. 利用吸收律$A+AB=A+B$，可将表达式$Y=AB+ABC$化简为（）。

A. $AB$　　B. $AB+C$　　C. $AB+C$

二、填空题

1. 逻辑代数的三种基本逻辑运算为_____、、。
2. 异或运算的表达式为 Y=A⊕B，当 A=1、B=0 时，输出 Y=；当 A=1、B=1 时，输出 Y=。
3. 十进制数 27 对应的二进制数为_____，对应的 8421BCD 码为_____。
4. 逻辑函数$Y=AB+AC$的最简与非-与非式为_____。
5. 卡诺图中，相邻方格的最小项仅有_____个变量不同，其余变量完全相同。
6. n 变量逻辑函数，共有_____个最小项，全部最小项的和恒等于_____。
7. 同或运算与异或运算的关系是_____。
8. 公式化简法中，配项法的核心理论依据是_____律。
9. 卡诺图合并相邻 1 方格时，每个圈的方格数量必须为_____。

10. 8421BCD 码逻辑电路中，约束项的编号为_____。

三、化简题（用公式法或卡诺图法化简为最简与-或式）

1. $Y = AB + B + BCD$
2. $Y = ABC + ABC + ABC + ABC$
3. $Y = \sum m(0,2,4,6,8,10) + \sum d(12,13,14,15)$
4. $Y = AB + AC + BC + CD + D$
5. $Y = AC + ABC + BC + ABC$

四、综合题

1. 写出二输入同或运算的真值表、逻辑表达式，并画出符合国标的逻辑符号。
2. 已知逻辑函数$Y = AB + AB$，用与非门实现该逻辑功能，写出转换后的与非-与非表达式，并画出对应的逻辑电路图。
3. 某三人表决电路，规则为：多数人同意则输出通过（Y=1），否则输出不通过（Y=0）。列出真值表，写出逻辑表达式，并化简为最简与-或式。
4. 用卡诺图化简四变量逻辑函数 Y=∑m(0,1,2,3,4,6,8,9,10,11,14)，写出最简与-或式。